# N-GOOD POLYNOMIALS

MARIO HERNÁNDEZ M.

Abstract. We prove that every integer polynomial is n-good if and only if n $\geq$ 3. The note explains the modular reduction, the finite checks for n=3 and n=4, and the odd-prime argument based on rootless quadratic polynomials over finite fields. The attached Lean file formalizes the reduction from modular witnesses to integer witnesses, the bad cases n=1 and n=2, the complete residue-class verifications for n=3 and n=4, the odd-prime analysis, and the final classification theorem.

0.1. **$n$-Good Polynomials.** We say that an integer polynomial $P$ is $n$-good if there exists a quadratic polynomial $Q \in \mathbf{Z}[x]$ such that

$$n \nmid Q(k)\big(P(k) + Q(k)\big)$$

for every integer $k$. In Lean this condition is encoded by `IntFunction NGood`, `PolynomialNGood`, and `AllPolynomialsNGood`.

**Theorem 0.1.** *Every polynomial with integer coefficients is $n$-good if and only if*

$$n \geq 3.$$

*Proof.* The cases $n = 1$ and $n = 2$ are impossible. For $n = 1$, divisibility by 1 is automatic, so no witness can exist. For $n = 2$, take $P(x) = 1$; then $Q(k)(Q(k) + 1)$ is always even, so the product is always divisible by 2. These obstructions are formalized in `not_allPolynomialsNGood_one` and `not_allPolynomialsNGood_two`.

The positive direction starts with a monotonicity observation: if $d \mid n$ and every polynomial is $d$-good, then every polynomial is $n$-good, because non-divisibility by $d$ implies non-divisibility by $n$. This is the content of the lifting lemmas `of_dvd`, `of_dvd`, and `of_dvd`. Hence it is enough to prove the claim for $n = 3$, $n = 4$, and every odd prime $p \geq 5$.

For $n = 3$ and $n = 4$, one may reduce modulo $n$ and work with polynomial functions on $\mathbf{Z}/n\mathbf{Z}$. The Lean file introduces the modular predicate `GoodModProduct` and the reduction map `polynomialModFn`, then lifts any modular quadratic witness back to an integer quadratic witness through `polynomialNGood_of_goodModProduct`. The residue class checks themselves are finite and are formalized by `goodModProduct_`

*Date*: April 18, 2026.

2020 *Mathematics Subject Classification.* 11C08, 11T06.

*Key words and phrases.* polynomial congruences, finite fields, quadratic polynomials, Lean 4.

`zmod_three`, `goodModProduct_zmod_four`, `allPolynomials NGood_three`, and `allPolynomialsNGood_four`.

It remains to handle an odd prime $p \geq 5$. Reduce $P$ modulo $p$, and view it as a polynomial function $f : \mathbf{F}_p \to \mathbf{F}_p$. We split into three cases, exactly as in the Lean proof.

First, if $f \equiv 0$, it is enough to choose a quadratic with no root in $\mathbf{F}_p$; for instance $q(x) = x^2 - u$, where $u$ is a fixed nonsquare. Then $q(x) \neq 0$ for every $x$, so

$$q(x)(q(x) + f(x)) = q(x)^2 \neq 0$$

for all $x \in \mathbf{F}_p$.

Second, suppose $f$ has both a zero and a nonzero value. Fix $z$ with $f(z) = 0$, and choose $x_0$ with $f(x_0) \neq 0$. Also choose a nonzero value $y_0$ such that $y_0 \neq -f(x_0)$. Consider the family of rootless quadratics $q$ satisfying $q(x_0) = y_0$. If every such $q$ were bad, we could assign to it some point $x$ such that $q(x) = -f(x)$. That point cannot be $x_0$, by the choice of $y_0$, and it cannot be $z$, because a rootless quadratic never vanishes. Thus every bad quadratic produces a point in $\mathbf{F}_p \setminus \{x_0, z\}$. For each fixed $x \neq x_0, z$, the condition $q(x) = -f(x)$ cuts the family down to at most $(p+1)/2$ possibilities. Since there are only $p-2$ such points, double counting would give

$$p \cdot \frac{p-1}{2} \leq (p-2) \cdot \frac{p+1}{2},$$

which is false for odd $p$. Hence some rootless quadratic in the family avoids $-f(x)$ everywhere.

Third, assume that $f$ is nowhere zero. If $f$ misses some nonzero value $a$, then the constant quadratic $q(x) = -a$ already works: it is never zero and $q(x) + f(x) = f(x) - a \neq 0$ for every $x$. So the only difficult case is when $f$ is surjective onto $\mathbf{F}_p^\times$.

In that last case, choose for each nonzero $y$ one preimage $x$ with $f(x) = y$. Since $|\mathbf{F}_p^\times| = p - 1 < p = |\mathbf{F}_p|$, this choice cannot cover all of $\mathbf{F}_p$; let $x_{\text{extra}}$ be a point that is not selected. Now fix a nonsquare $n \in \mathbf{F}_p^\times$, and set

$$y_0 = -f(x_{\text{extra}}), \qquad u = \frac{n}{f(x_{\text{extra}})}, \qquad x_0 \text{ chosen so that } f(x_0) = u.$$

Then $y_0(-u) = n$ is a nonsquare, so in particular $y_0 \neq -f(x_0)$. Consider again the family of rootless quadratics $q$ with $q(x_0) = y_0$.

If every such $q$ were bad, then each one would either be bad at the special point $x_{\text{extra}}$, or else at some point of the form $x = \pi(y)$, where $y \in \mathbf{F}_p^\times \setminus \{u\}$ and $\pi(y)$ is the chosen preimage of $y$. For a fixed $y$, the number of quadratics with $q(\pi(y)) = -y$ depends on whether $y_0(-y)$ is a square or a nonsquare: in the square case there are at most $(p-1)/2$, and in the nonsquare case at most $(p+1)/2$. Moreover, because multiplication by $y_0$ permutes $\mathbf{F}_p^\times$ and $y_0(-u) = n$ is a nonsquare, the values $y_0(-y)$ split as follows when $y$ runs

through $\mathbf{F}_p^\times \setminus \{u\}$:

$$\frac{p-1}{2} \text{ of them are squares, and } \frac{p-3}{2} \text{ are nonsquares.}$$

The special point $x_{\text{extra}}$ contributes at most $(p-1)/2$ more quadratics. Therefore double counting would force

$$p \cdot \frac{p-1}{2} \le \frac{p-1}{2} + \frac{p-1}{2} \cdot \frac{p-1}{2} + \frac{p-3}{2} \cdot \frac{p+1}{2},$$

and this inequality is again false for odd $p$. Hence some rootless quadratic $q$ in the family is not bad at any point. In other words,

$$q(x)(q(x) + f(x)) \neq 0 \qquad \text{for all } x \in \mathbf{F}_p.$$

Finally, lift $q$ to an integer quadratic $Q$. Since $Q(k)$ and $P(k)$ reduce to $q(\bar{k})$ and $f(\bar{k})$ modulo $p$, the product $Q(k)(P(k)+Q(k))$ is never divisible by $p$, and therefore not divisible by any multiple of $p$. Together with the cases 3 and 4, this proves that every polynomial is $n$-good for every $n \ge 3$. □

The Lean development formalizes the odd-prime analysis through `goodModFunction_zmod_odd_prime_zero`, `goodModFunction_zmod_odd_prime_has_zero`, `goodModFunction_zmod_of_not_surjective_nonzero`, `goodModFunction_zmod_odd_prime_nowhere_zero_surjective`, `goodModProduct_zmod_odd_prime`, and `allPolynomialsNGood_odd_prime`, and concludes with the classification theorem `allPolynomialsNGood_iff_of_pos`. It also keeps the interpolation lemma `card_quadCoeff_eval_eq` as a reusable finite-field ingredient.

## Lean Glossary

```
IntFunctionNGood
def IntFunctionNGood (n : ℕ) (f : ℤ → ℤ) : Prop
PolynomialNGood
def PolynomialNGood (n : ℕ) (P : Polynomial ℤ) : Prop
AllPolynomialsNGood
def AllPolynomialsNGood (n : ℕ) : Prop
not_allPolynomialsNGood_one
lemma not_allPolynomialsNGood_one : ¬AllPolynomialsNGood 1
not_allPolynomialsNGood_two
lemma not_allPolynomialsNGood_two : ¬AllPolynomialsNGood 2
of_dvd
Biblioteca.Demonstrations.IntFunctionNGood.of_dvd
of_dvd
Biblioteca.Demonstrations.PolynomialNGood.of_dvd
of_dvd
Biblioteca.Demonstrations.AllPolynomialsNGood.of_dvd
```

GoodModProduct

```
def GoodModProduct {R : Type*} [Semiring R] [DecidableEq
↪  R] [Fintype R] (f : R → R) : Prop
```

polynomialModFn

```
def polynomialModFn (n : ℕ) (P : Polynomial ℤ) : ZMod n →
↪  ZMod n
```

polynomialNGood_of_goodModProduct

```
lemma polynomialNGood_of_goodModProduct {n : ℕ} [NeZero n]
↪  (P : Polynomial ℤ) (hmod : GoodModProduct
↪  (polynomialModFn n P)) : PolynomialNGood n P
```

goodModProduct_zmod_three

```
lemma goodModProduct_zmod_three : ∀ f : ZMod 3 → ZMod 3,
↪  GoodModProduct f
```

goodModProduct_zmod_four

```
lemma goodModProduct_zmod_four : ∀ f : ZMod 4 → ZMod 4,
↪  GoodModProduct f
```

allPolynomialsNGood_three

```
lemma allPolynomialsNGood_three : AllPolynomialsNGood 3
```

allPolynomialsNGood_four

```
lemma allPolynomialsNGood_four : AllPolynomialsNGood 4
```

goodModFunction_zmod_odd_prime_zero

```
lemma goodModFunction_zmod_odd_prime_zero {p : ℕ} [Fact
↪  p.Prime] (hp2 : p ≠ 2) : GoodModFunction (fun _ : ZMod
↪  p => 0)
```

goodModFunction_zmod_odd_prime_has_zero

```
lemma goodModFunction_zmod_odd_prime_has_zero {p : ℕ}
↪  [Fact p.Prime] (hp2 : p ≠ 2) {f : ZMod p → ZMod p} (hz
↪  : ∃ z, f z = 0) (hnot0 : ∃ x, f x ≠ 0) :
↪  GoodModFunction f
```

goodModFunction_zmod_of_not_surjective_nonzero

```
lemma goodModFunction_zmod_of_not_surjective_nonzero {p :
↪  ℕ} [Fact p.Prime] {f : ZMod p → ZMod p} (hnsurj : ¬ ∀
↪  y : ZMod p, y ≠ 0 → ∃ x : ZMod p, f x = y) :
↪  GoodModFunction f
```

goodModFunction_zmod_odd_prime_nowhere_zero_surjective

```
lemma goodModFunction_zmod_odd_prime_nowhere_zero_surject
↪  ive {p : ℕ} [Fact p.Prime] (hp2 : p ≠ 2) (_hp3 : p ≠
↪  3) {f : ZMod p → ZMod p} (hnozero : ∀ x : ZMod p, f x ≠
↪  0) (hsurj : ∀ y : ZMod p, y ≠ 0 → ∃ x : ZMod p, f x =
↪  y) : GoodModFunction f
```

goodModProduct_zmod_odd_prime

```
lemma goodModProduct_zmod_odd_prime {p : ℕ} [Fact p.Prime]
↪  (hp2 : p ≠ 2) : ∀ f : ZMod p → ZMod p, GoodModProduct f
```

allPolynomialsNGood_odd_prime

```
lemma allPolynomialsNGood_odd_prime {p : ℕ} [Fact p.Prime]
 ↪  (hp2 : p ≠ 2) : AllPolynomialsNGood p
allPolynomialsNGood_iff_of_pos
theorem allPolynomialsNGood_iff_of_pos {n : ℕ} (hn : 0 <
 ↪  n) : AllPolynomialsNGood n ↔ 3 ≤ n
card_quadCoeff_eval_eq
lemma card_quadCoeff_eval_eq {p : ℕ} [Fact p.Prime] (x y :
 ↪  ZMod p) : Fintype.card {v : QuadCoeff p //
 ↪  quadCoeffEval v x = y} = p ^ 2
```

## Anexo

**Biblioteca/Demonstrations/Demo_20260403_172608_n_good_polynomials.lean.**

```
import Mathlib.RingTheory.Polynomial.Basic
import Mathlib.Algebra.Ring.Int.Parity
import Mathlib.Data.Fintype.Pi
import Mathlib.Data.Fintype.Card
import Mathlib.Data.ZMod.Basic
import Mathlib.Data.ZMod.Units
import Mathlib.LinearAlgebra.Vandermonde
import Mathlib.FieldTheory.Finite.Basic
import Mathlib.GroupTheory.SpecificGroups.Cyclic
import Mathlib.NumberTheory.JacobiSum.Basic
import Mathlib.NumberTheory.LegendreSymbol.QuadraticChar.
 ↪  Basic
import Mathlib.Tactic

namespace Biblioteca.Demonstrations

set_option linter.style.longLine false
set_option linter.unnecessarySeqFocus false
set_option linter.unnecessarySimpa false
set_option linter.unreachableTactic false
set_option linter.unusedSimpArgs false
set_option linter.unusedTactic false
set_option linter.unusedVariables false

open Polynomial

/-- The value of the quadratic polynomial with
 ↪  coefficients `a, b, c` at `x`. -/
def quadValue {R : Type*} [Semiring R] (a b c x : R) : R :=
  a * x ^ 2 + b * x + c

/-- A function `f : ℤ → ℤ` is `n`-good if some integer
 ↪  quadratic witness avoids
divisibility by `n` at every integer input. -/
```

```
def IntFunctionNGood (n : ℕ) (f : ℤ → ℤ) : Prop :=
  ∃ a b c : ℤ,
    a ≠ 0 ∧
      ∀ k : ℤ,
        ¬((n : ℤ) ∣ quadValue a b c k * (f k + quadValue a
         ↪  b c k))

/-- An integer polynomial is `n`-good when its evaluation
 ↪  function is. -/
def PolynomialNGood (n : ℕ) (P : Polynomial ℤ) : Prop :=
  IntFunctionNGood n fun k => P.eval k

/-- Every integer polynomial is `n`-good. -/
def AllPolynomialsNGood (n : ℕ) : Prop :=
  ∀ P : Polynomial ℤ, PolynomialNGood n P

/-- If a quadratic witness already avoids divisibility by
 ↪  a divisor `m`, then it
also avoids divisibility by any multiple `n`. -/
lemma IntFunctionNGood.of_dvd {m n : ℕ} (hmn : m ∣ n) {f :
 ↪  ℤ → ℤ}
    (hf : IntFunctionNGood m f) : IntFunctionNGood n f :=
     ↪  by
  rcases hf with ⟨a, b, c, ha, hgood⟩
  refine ⟨a, b, c, ha, ?_⟩
  intro k hk
  exact hgood k (dvd_trans (by exact_mod_cast hmn) hk)

lemma PolynomialNGood.of_dvd {m n : ℕ} (hmn : m ∣ n) {P :
 ↪  Polynomial ℤ}
    (hP : PolynomialNGood m P) : PolynomialNGood n P :=
  IntFunctionNGood.of_dvd hmn hP

lemma AllPolynomialsNGood.of_dvd {m n : ℕ} (hmn : m ∣ n)
 ↪  (hm : AllPolynomialsNGood m) :
    AllPolynomialsNGood n := by
  intro P
  exact PolynomialNGood.of_dvd hmn (hm P)

lemma not_allPolynomialsNGood_one : ¬AllPolynomialsNGood 1
 ↪  := by
  intro h
  rcases h (0 : Polynomial ℤ) with ⟨a, b, c, ha, hgood⟩
  exact hgood 0 (by simp [quadValue])

lemma not_allPolynomialsNGood_two : ¬AllPolynomialsNGood 2
 ↪  := by
  intro h
```

```
  rcases h (1 : Polynomial ℤ) with ⟨a, b, c, ha, hgood⟩
  have hzero := hgood 0
  apply hzero
  simpa [quadValue, add_comm, add_left_comm, add_assoc,
   ↪  mul_comm, mul_left_comm, mul_assoc] using
    even_iff_two_dvd.mp (Int.even_mul_succ_self c)

/-- Modulo `n`, the quadratic `a x^2 + b x + c` makes the
 ↪  relevant product
nonzero at every input. We allow the quadratic coefficient
 ↪  to vanish modulo `n`;
later we lift it to an honest integer quadratic by adding
 ↪  a multiple of `n` to
the leading coefficient. -/
def GoodModProduct {R : Type*} [Semiring R] [DecidableEq
 ↪  R] [Fintype R] (f : R → R) : Prop :=
  ∃ a b c : R,
    ∀ x ∈ (Finset.univ : Finset R),
      quadValue a b c x * (quadValue a b c x + f x) ≠ 0

/-- In a ring without zero divisors, the stronger
 ↪  factorwise condition implies
the product condition. -/
def GoodModFunction {R : Type*} [Semiring R] [DecidableEq
 ↪  R] [Fintype R] (f : R → R) : Prop :=
  ∃ a b c : R,
    ∀ x ∈ (Finset.univ : Finset R), quadValue a b c x ≠ 0 ∧
     ↪  quadValue a b c x + f x ≠ 0

instance {R : Type*} [Semiring R] [DecidableEq R] [Fintype
 ↪  R] (f : R → R) :
    Decidable (GoodModProduct f) := by
  unfold GoodModProduct
  infer_instance

instance {R : Type*} [Semiring R] [DecidableEq R] [Fintype
 ↪  R] (f : R → R) :
    Decidable (GoodModFunction f) := by
  unfold GoodModFunction
  infer_instance

lemma GoodModFunction.to_product {R : Type*} [Semiring R]
 ↪  [NoZeroDivisors R]
    [DecidableEq R] [Fintype R] {f : R → R} (h :
     ↪  GoodModFunction f) : GoodModProduct f := by
  rcases h with ⟨a, b, c, hgood⟩
  refine ⟨a, b, c, ?_⟩
```

```
  intro x hx
  rcases hgood x hx with ⟨hq, hqf⟩
  exact mul_ne_zero hq hqf

lemma goodModFunction_of_constant {R : Type*} [Semiring R]
 ↪  [DecidableEq R] [Fintype R]
    {f : R → R} {c : R} (hc : c ≠ 0) (havoid : ∀ x : R, c +
     ↪  f x ≠ 0) :
    GoodModFunction f := by
  refine ⟨0, 0, c, ?_⟩
  intro x hx
  constructor
  · simp [quadValue, hc]
  · simpa [quadValue] using havoid x

lemma goodModFunction_of_missing_nonzero {R : Type*} [Ring
 ↪  R] [DecidableEq R] [Fintype R]
    {f : R → R} (hmiss : ∃ y : R, y ≠ 0 ∧ ∀ x : R, f x ≠ y) :
    GoodModFunction f := by
  rcases hmiss with ⟨y, hy, hmissy⟩
  refine goodModFunction_of_constant (c := -y)
   ↪  (neg_ne_zero.mpr hy) ?_
  intro x
  have hxy : f x ≠ y := hmissy x
  simpa [sub_eq_add_neg, add_comm, add_left_comm,
   ↪  add_assoc] using sub_ne_zero.mpr hxy

lemma goodModFunction_zmod_of_not_surjective_nonzero {p :
 ↪  ℕ} [Fact p.Prime]
    {f : ZMod p → ZMod p}
    (hnsurj : ¬ ∀ y : ZMod p, y ≠ 0 → ∃ x : ZMod p, f x =
     ↪  y) :
    GoodModFunction f := by
  push Not at hnsurj
  rcases hnsurj with ⟨y, hy, hmiss⟩
  exact goodModFunction_of_missing_nonzero ⟨y, hy, fun x
   ↪  => hmiss x⟩

/-- The reduction of an integer polynomial modulo `n` as a
 ↪  function on `ZMod n`. -/
def polynomialModFn (n : ℕ) (P : Polynomial ℤ) : ZMod n →
 ↪  ZMod n :=
  fun x => P.eval₂ (Int.castRingHom (ZMod n)) x

lemma polynomialNGood_of_goodModProduct {n : ℕ} [NeZero n]
 ↪  (P : Polynomial ℤ)
```

```
    (hmod : GoodModProduct (polynomialModFn n P)) :
     ↪  PolynomialNGood n P := by
  have hn : 0 < n := Nat.pos_of_ne_zero (NeZero.ne n)
  rcases hmod with ⟨a, b, c, hgood⟩
  let A : ℤ := a.val + n
  let B : ℤ := b.val
  let C : ℤ := c.val
  refine ⟨A, B, C, ?_, ?_⟩
  · dsimp [A]
    omega
  · intro k hk
    have hk0 : (((quadValue A B C k) * (P.eval k +
     ↪  quadValue A B C k) : ℤ) : ZMod n) = 0 := by
      simpa using (ZMod.intCast_zmod_eq_zero_iff_dvd
        ((quadValue A B C k) * (P.eval k + quadValue A B C
         ↪  k)) n).2 hk
    have hquad :
        ((quadValue A B C k : ℤ) : ZMod n) = quadValue a b
         ↪  c (k : ZMod n) := by
      simp [A, B, C, quadValue, pow_two]
    have hpoly : ((P.eval k : ℤ) : ZMod n) =
     ↪  polynomialModFn n P (k : ZMod n) := by
      simp [polynomialModFn]
    have hprod :
        quadValue a b c (k : ZMod n) *
            (quadValue a b c (k : ZMod n) + polynomialModFn
             ↪  n P (k : ZMod n)) = 0 := by
      calc
        quadValue a b c (k : ZMod n) *
            (quadValue a b c (k : ZMod n) + polynomialModFn
             ↪  n P (k : ZMod n))
            = (((quadValue A B C k : ℤ) : ZMod n) *
                (((quadValue A B C k : ℤ) : ZMod n) +
                 ↪  ((P.eval k : ℤ) : ZMod n))) := by
                  simp [hquad, hpoly]
        _ = ((((quadValue A B C k) * (P.eval k + quadValue
         ↪  A B C k) : ℤ) : ZMod n)) := by
              norm_num [mul_add, add_mul, add_comm,
               ↪  add_left_comm, add_assoc, mul_comm,
                mul_left_comm, mul_assoc]
        _ = 0 := hk0
    exact hgood (k : ZMod n) (by simp) hprod

set_option linter.style.nativeDecide false in
lemma goodModProduct_zmod_three :
    ∀ f : ZMod 3 → ZMod 3, GoodModProduct f := by
  native_decide
```

```
set_option linter.style.nativeDecide false in
lemma goodModProduct_zmod_four :
    ∀ f : ZMod 4 → ZMod 4, GoodModProduct f := by
  native_decide

lemma allPolynomialsNGood_three : AllPolynomialsNGood 3 :=
 ↪  by
  intro P
  exact polynomialNGood_of_goodModProduct P
   ↪  (goodModProduct_zmod_three (polynomialModFn 3 P))

lemma allPolynomialsNGood_four : AllPolynomialsNGood 4 :=
 ↪  by
  intro P
  exact polynomialNGood_of_goodModProduct P
   ↪  (goodModProduct_zmod_four (polynomialModFn 4 P))

set_option linter.style.nativeDecide false in
lemma goodModProduct_zmod_five :
    ∀ f : ZMod 5 → ZMod 5, GoodModProduct f := by
  native_decide

lemma allPolynomialsNGood_five : AllPolynomialsNGood 5 :=
 ↪  by
  intro P
  exact polynomialNGood_of_goodModProduct P
   ↪  (goodModProduct_zmod_five (polynomialModFn 5 P))

lemma goodModFunction_zmod_odd_prime_zero {p : ℕ} [Fact
 ↪  p.Prime] (hp2 : p ≠ 2) :
    GoodModFunction (fun _ : ZMod p => 0) := by
  have hchar : ringChar (ZMod p) ≠ 2 := by
    simpa [ZMod.ringChar_zmod_n p] using hp2
  obtain ⟨u, hu⟩ := FiniteField.exists_nonsquare (F :=
   ↪  ZMod p) hchar
  have hrootless : ∀ x : ZMod p, quadValue (1 : ZMod p) 0
   ↪  (-u) x ≠ 0 := by
    intro x hzero
    apply hu
    refine ⟨x, ?_⟩
    have hx : x ^ 2 = u := by
      simpa using eq_neg_of_add_eq_zero_left hzero
    simpa [pow_two] using hx.symm
  refine ⟨1, 0, -u, ?_⟩
  intro x hx
  constructor
  · exact hrootless x
  · simpa using hrootless x
```

```
lemma exists_quadraticChar_pos_neg_of_surjective_nonzero
 ↪  {p : ℕ} [Fact p.Prime] (hp2 : p ≠ 2)
    {f : ZMod p → ZMod p}
    (hsurj : ∀ y : ZMod p, y ≠ 0 → ∃ x : ZMod p, f x = y) :
    (∃ x : ZMod p, quadraticChar (ZMod p) (-f x) = 1) ∧
      (∃ x : ZMod p, quadraticChar (ZMod p) (-f x) = -1) :=
       ↪  by
  have hchar : ringChar (ZMod p) ≠ 2 := by
    simpa [ZMod.ringChar_zmod_n p] using hp2
  constructor
  · rcases hsurj (-1) (neg_ne_zero.mpr one_ne_zero) with
   ↪  ⟨x, hx⟩
    refine ⟨x, ?_⟩
    simpa [hx] using (quadraticChar_one : quadraticChar
     ↪  (ZMod p) (1 : ZMod p) = 1)
  · obtain ⟨u, hu⟩ := quadraticChar_exists_neg_one (F :=
   ↪  ZMod p) hchar
    have hu0 : u ≠ 0 := by
      intro hu0
      have : quadraticChar (ZMod p) (0 : ZMod p) = -1 := by
       ↪  simpa [hu0] using hu
      simp at this
    rcases hsurj (-u) (neg_ne_zero.mpr hu0) with ⟨x, hx⟩
    refine ⟨x, ?_⟩
    simpa [hx] using hu

abbrev QuadCoeff (p : ℕ) := Fin 3 → ZMod p

def quadCoeffEval {p : ℕ} (v : QuadCoeff p) (x : ZMod p) :
 ↪  ZMod p :=
  ∑ i : Fin 3, v i * x ^ (i : ℕ)

def quadDiscr {p : ℕ} (v : QuadCoeff p) : ZMod p :=
  v 1 ^ 2 - 4 * v 2 * v 0

/-- Quadratic coefficient vectors with nonzero leading
 ↪  term and nonsquare discriminant. -/
abbrev RootlessQuad (p : ℕ) :=
  {v : QuadCoeff p // v 2 ≠ 0 ∧ ¬IsSquare (quadDiscr v)}

lemma quadDiscr_eq_square_sub {p : ℕ} (v : QuadCoeff p) (x
 ↪  : ZMod p) :
    quadDiscr v = (2 * v 2 * x + v 1) ^ 2 - 4 * v 2 *
     ↪  quadCoeffEval v x := by
  simp [quadDiscr, quadCoeffEval, Fin.sum_univ_three]
  ring
```

```
lemma rootlessQuad_eval_ne_zero {p : ℕ} (q : RootlessQuad
 ↪  p) (x : ZMod p) :
    quadCoeffEval q.1 x ≠ 0 := by
  intro hx
  apply q.2.2
  refine ⟨2 * q.1 2 * x + q.1 1, ?_⟩
  have h := quadDiscr_eq_square_sub q.1 x
  simp [hx] at h
  simpa [pow_two] using h

lemma card_squareUnits_zmod_odd_prime {p : ℕ} [Fact
 ↪  p.Prime] (hp2 : p ≠ 2) :
    Fintype.card {u : (ZMod p)ˣ // IsSquare u} = (p - 1) /
     ↪  2 := by
  classical
  let e : {u : (ZMod p)ˣ // IsSquare u} ≃
      (powMonoidHom 2 : (ZMod p)ˣ →* (ZMod p)ˣ).range := {
    toFun := fun u => ⟨u.1, by
      rcases u.2 with ⟨v, hv⟩
      exact ⟨v, by simpa [pow_two, eq_comm] using hv⟩⟩
    invFun := fun u => ⟨u.1, by
      rcases u.2 with ⟨v, hv⟩
      exact ⟨v, by simpa [pow_two, eq_comm] using hv⟩⟩
    left_inv := by intro u; cases u; rfl
    right_inv := by intro u; cases u; rfl
  }
  have hrange := IsCyclic.card_powMonoidHom_range ((ZMod
   ↪  p)ˣ) 2
  have hgcd : (p - 1).gcd 2 = 2 := by
    exact Nat.gcd_eq_right ((Fact.out : Nat.Prime
     ↪  p).even_sub_one hp2).two_dvd
  calc
    Fintype.card {u : (ZMod p)ˣ // IsSquare u}
        = Fintype.card ((powMonoidHom 2 : (ZMod p)ˣ →*
         ↪  (ZMod p)ˣ).range) := by
            exact Fintype.card_congr e
    _ = (p - 1) / 2 := by
      rw [← Nat.card_eq_fintype_card, hrange,
       ↪  Nat.card_eq_fintype_card, ZMod.card_units p]
      simp [hgcd]

lemma card_nonsquare_zmod_odd_prime {p : ℕ} [Fact p.Prime]
 ↪  (hp2 : p ≠ 2) :
    Fintype.card {u : ZMod p // ¬IsSquare u} = (p - 1) / 2
     ↪  := by
  classical
  let e : {u : ZMod p // ¬IsSquare u} ≃ {u : (ZMod p)ˣ //
   ↪  ¬IsSquare u} := {
```

```
  toFun := fun u => ⟨Units.mk0 u.1 (by
    intro hu0
    apply u.2
    simpa [hu0] using (IsSquare.zero : IsSquare (0 : ZMod
     ↪  p))), by
      intro hs
      apply u.2
      rcases hs with ⟨v, hv⟩
      refine ⟨(v : ZMod p), ?_⟩
      simpa using congrArg (fun z : (ZMod p)ˣ => (z :
       ↪  ZMod p)) hv⟩
  invFun := fun u => ⟨u.1, by
    intro hs
    rcases hs with ⟨a, ha⟩
    have ha0 : a ≠ 0 := by
      intro ha0
      have : ((u.1 : (ZMod p)ˣ) : ZMod p) = 0 := by
        simpa [ha0] using ha
      exact Units.ne_zero u.1 this
    apply u.2
    refine ⟨Units.mk0 a ha0, ?_⟩
    ext
    simpa [pow_two] using ha⟩
  left_inv := by intro u; cases u; rfl
  right_inv := by
    intro u
    cases u
    ext <;> rfl
}
have hcompl :
    Fintype.card {u : (ZMod p)ˣ // ¬IsSquare u} =
      Fintype.card (ZMod p)ˣ - Fintype.card {u : (ZMod
       ↪  p)ˣ // IsSquare u} :=
  Fintype.card_subtype_compl (IsSquare : (ZMod p)ˣ →
   ↪  Prop)
have hhalf :
    Fintype.card (ZMod p)ˣ - Fintype.card {u : (ZMod p)ˣ
     ↪  // IsSquare u} = (p - 1) / 2 := by
  rw [ZMod.card_units p, card_squareUnits_zmod_odd_prime
   ↪  hp2]
  obtain ⟨k, hk⟩ := ((Fact.out : Nat.Prime
   ↪  p).even_sub_one hp2).two_dvd
  omega
calc
  Fintype.card {u : ZMod p // ¬IsSquare u}
      = Fintype.card {u : (ZMod p)ˣ // ¬IsSquare u} :=
       ↪  Fintype.card_congr e
  _ = (p - 1) / 2 := by
```

```
      rw [hcompl]
      exact hhalf

lemma card_nonzero_square_zmod_odd_prime {p : ℕ} [Fact
 ↪  p.Prime] (hp2 : p ≠ 2) :
    Fintype.card {u : ZMod p // IsSquare u ∧ u ≠ 0} = (p -
     ↪  1) / 2 := by
  classical
  let e : {u : ZMod p // IsSquare u ∧ u ≠ 0} ≃ {u : (ZMod
   ↪  p)ˣ // IsSquare u} := {
    toFun := fun u => ⟨Units.mk0 u.1 u.2.2, by
      rcases u.2.1 with ⟨v, hv⟩
      have hv0 : v ≠ 0 := by
        intro hv0
        have : (u.1 : ZMod p) = 0 := by
          simpa [hv0] using hv
        exact u.2.2 this
      refine ⟨Units.mk0 v hv0, ?_⟩
      ext
      simpa [pow_two] using hv⟩
    invFun := fun u => ⟨u.1, by
      constructor
      · rcases u.2 with ⟨v, hv⟩
        refine ⟨(v : ZMod p), ?_⟩
        simpa [pow_two] using congrArg (fun z : (ZMod p)ˣ
         ↪  => (z : ZMod p)) hv
      · exact Units.ne_zero u.1⟩
    left_inv := by
      intro u
      cases u
      rfl
    right_inv := by
      intro u
      cases u
      ext <;> rfl
  }
  calc
    Fintype.card {u : ZMod p // IsSquare u ∧ u ≠ 0}
        = Fintype.card {u : (ZMod p)ˣ // IsSquare u} :=
         ↪  Fintype.card_congr e
    _ = (p - 1) / 2 := card_squareUnits_zmod_odd_prime hp2

lemma card_rootlessQuad_eval_eq {p : ℕ} [Fact p.Prime]
 ↪  (hp2 : p ≠ 2)
    (x y : ZMod p) (hy : y ≠ 0) :
    Fintype.card {q : RootlessQuad p // quadCoeffEval q.1
     ↪  x = y} = p * ((p - 1) / 2) := by
  classical
```

```
have h4 : (4 : ZMod p) ≠ 0 := by
  intro hzero
  have hpdvd4 : p ∣ 4 := by
    simpa using (ZMod.natCast_eq_zero_iff 4 p).mp hzero
  have hpdvd2pow : p ∣ 2 ^ 2 := by simpa [pow_two] using
   ↪  hpdvd4
  have hpdvd2 : p ∣ 2 := (Fact.out : Nat.Prime
   ↪  p).dvd_of_dvd_pow hpdvd2pow
  have hp_le2 : p ≤ 2 := Nat.le_of_dvd (by decide : 0 <
   ↪  2) hpdvd2
  have hp_ge2 : 2 ≤ p := (Fact.out : Nat.Prime p).two_le
  omega
have hden : (4 : ZMod p) * y ≠ 0 := mul_ne_zero h4 hy
let e : {q : RootlessQuad p // quadCoeffEval q.1 x = y} ≃
    ZMod p × {u : ZMod p // ¬IsSquare u} := {
  toFun := fun q => (2 * q.1.1 2 * x + q.1.1 1,
   ↪  ⟨quadDiscr q.1.1, q.1.2.2⟩)
  invFun := fun mu =>
    let a : ZMod p := (mu.1 ^ 2 - mu.2.1) / ((4 : ZMod p)
     ↪  * y)
    let b : ZMod p := mu.1 - 2 * a * x
    let c : ZMod p := y - b * x - a * x ^ 2
    ⟨⟨![c, b, a], by
        have ha0 : a ≠ 0 := by
          intro ha0
          apply mu.2.2
          refine ⟨mu.1, ?_⟩
          have hdiv : (mu.1 ^ 2 - mu.2.1) / ((4 : ZMod p)
           ↪  * y) = 0 := by
            simpa [a] using ha0
          have hnum : mu.1 ^ 2 - mu.2.1 = 0 := by
            rcases (div_eq_zero_iff).mp hdiv with hnum |
             ↪  hbad
            · exact hnum
            · exact (hden hbad).elim
          simpa [pow_two, sub_eq_zero] using
           ↪  (sub_eq_zero.mp hnum).symm
        have hdisc : quadDiscr ![c, b, a] = mu.2.1 := by
          simp [quadDiscr, a, b, c]
          field_simp [hden]
          ring_nf
        exact ⟨ha0, by simpa [hdisc] using mu.2.2⟩⟩,
      by
        simp [quadCoeffEval, Fin.sum_univ_three, a, b, c]
         ring⟩
  left_inv := by
    intro q
    let v : QuadCoeff p := q.1.1
```

```
    have hq : quadCoeffEval v x = y := q.2
    have hdisc :
        quadDiscr v = (2 * v 2 * x + v 1) ^ 2 - 4 * v 2 *
         ↪  y := by
      simpa [v, hq] using quadDiscr_eq_square_sub v x
    have ha :
        ((2 * v 2 * x + v 1) ^ 2 - quadDiscr v) / ((4 :
         ↪  ZMod p) * y) = v 2 := by
      rw [hdisc]
      field_simp [hden]
      ring
    have hb :
        2 * v 2 * x + v 1 -
            2 * (((2 * v 2 * x + v 1) ^ 2 - quadDiscr v)
             ↪  / ((4 : ZMod p) * y)) * x =
          v 1 := by
      simp [ha]
    have hc :
        y - v 1 * x - v 2 * x ^ 2 = v 0 := by
      calc
        y - v 1 * x - v 2 * x ^ 2 = quadCoeffEval v x - v
         ↪  1 * x - v 2 * x ^ 2 := by rw [hq]
        _ = v 0 := by
          simp [quadCoeffEval, Fin.sum_univ_three]
          ring
    ext i
    fin_cases i
    · change
        y -
            (2 * v 2 * x + v 1 -
               2 * (((2 * v 2 * x + v 1) ^ 2 - quadDiscr
                ↪  v) / (4 * y)) * x) *
              x -
            (((2 * v 2 * x + v 1) ^ 2 - quadDiscr v) / (4
             ↪  * y)) * x ^ 2 =
          v 0
      simpa [ha, hb] using hc
    · change
        2 * v 2 * x + v 1 -
            2 * (((2 * v 2 * x + v 1) ^ 2 - quadDiscr v)
             ↪  / (4 * y)) * x =
          v 1
      exact hb
    · change ((2 * v 2 * x + v 1) ^ 2 - quadDiscr v) / (4
     ↪  * y) = v 2
      exact ha
  right_inv := by
    intro mu
```

```
      have hdisc :
          quadDiscr
              ![y - (mu.1 - 2 * ((mu.1 ^ 2 - mu.2.1) / ((4
               ↪  : ZMod p) * y)) * x) * x -
                  ((mu.1 ^ 2 - mu.2.1) / ((4 : ZMod p) *
                   ↪  y)) * x ^ 2,
                mu.1 - 2 * ((mu.1 ^ 2 - mu.2.1) / ((4 :
                 ↪  ZMod p) * y)) * x,
                (mu.1 ^ 2 - mu.2.1) / ((4 : ZMod p) * y)] =
                 ↪  mu.2.1 := by
        simp [quadDiscr]
        field_simp [hden]
        ring
      ext
      · simp
      · simp [hdisc]
  }
  calc
    Fintype.card {q : RootlessQuad p // quadCoeffEval q.1
     ↪  x = y}
        = Fintype.card (ZMod p × {u : ZMod p // ¬IsSquare
         ↪  u}) := Fintype.card_congr e
    _ = p * ((p - 1) / 2) := by
      rw [Fintype.card_prod, ZMod.card,
       ↪  card_nonsquare_zmod_odd_prime hp2]

lemma card_nonzero_zmod {p : ℕ} [Fact p.Prime] :
    Fintype.card {x : ZMod p // x ≠ 0} = p - 1 := by
  classical
  calc
    Fintype.card {x : ZMod p // x ≠ 0}
        = Fintype.card (ZMod p) - Fintype.card {x : ZMod p
         ↪  // x = 0} := by
            simpa using
              (Fintype.card_subtype_compl (fun x : ZMod p
               ↪  => x = 0) :
                Fintype.card {x : ZMod p // ¬x = 0} =
                 Fintype.card (ZMod p) - Fintype.card {x :
                  ↪  ZMod p // x = 0})
    _ = p - 1 := by
      rw [ZMod.card]
      simp

lemma card_rootlessQuad {p : ℕ} [Fact p.Prime] (hp2 : p ≠
 ↪  2) :
    Fintype.card (RootlessQuad p) = p * ((p - 1) / 2) * (p
     ↪  - 1) := by
```

```
classical
let g : RootlessQuad p → {y : ZMod p // y ≠ 0} :=
  fun q => ⟨quadCoeffEval q.1 0,
   ↪  rootlessQuad_eval_ne_zero q 0⟩
have hfiber :
    ∀ y : {y : ZMod p // y ≠ 0},
      Fintype.card {q : RootlessQuad p // g q = y} =
        Fintype.card {q : RootlessQuad p //
         ↪  quadCoeffEval q.1 0 = y.1} := by
  intro y
  let e : {q : RootlessQuad p // g q = y} ≃ {q :
   ↪  RootlessQuad p // quadCoeffEval q.1 0 = y.1} := {
    toFun := fun q => ⟨q.1, congrArg Subtype.val q.2⟩
    invFun := fun q => ⟨q.1, Subtype.ext q.2⟩
    left_inv := by intro q; ext; rfl
    right_inv := by intro q; ext; rfl
  }
  exact Fintype.card_congr e
calc
  Fintype.card (RootlessQuad p)
      = Fintype.card (Σ y : {y : ZMod p // y ≠ 0}, {q :
       ↪  RootlessQuad p // g q = y}) := by
          exact Fintype.card_congr
           ↪  (Equiv.sigmaFiberEquiv g).symm
  _ = Σ y : {y : ZMod p // y ≠ 0}, Fintype.card {q :
   ↪  RootlessQuad p // g q = y} := by
        rw [Fintype.card_sigma]
  _ = Σ y : {y : ZMod p // y ≠ 0}, Fintype.card {q :
   ↪  RootlessQuad p // quadCoeffEval q.1 0 = y.1} := by
        simp [hfiber]
 _ = Σ _y : {y : ZMod p // y ≠ 0}, p * ((p - 1) / 2) := by
        refine Finset.sum_congr rfl ?_
        intro y hy
        exact card_rootlessQuad_eval_eq hp2 0 y.1 y.2
  _ = Fintype.card {y : ZMod p // y ≠ 0} * (p * ((p - 1)
   ↪  / 2)) := by
        rw [Finset.sum_const, nsmul_eq_mul]
        simpa using
          (show Fintype.card ↥(Finset.univ : Finset {y :
           ↪  ZMod p // y ≠ 0}) *
              (p * ((p - 1) / 2)) =
               Fintype.card {y : ZMod p // y ≠ 0} * (p *
                ↪  ((p - 1) / 2)) by
              simp)
  _ = p * ((p - 1) / 2) * (p - 1) := by
        rw [card_nonzero_zmod]
        ring
```

```
def squareWitnessMap {p : ℕ} [Fact p.Prime] (m : ZMod p)
↪  (r : {x : ZMod p // x ≠ 0}) : ZMod p :=
  (r.1 + m / r.1) / 2

lemma squareWitnessMap_sq_sub {p : ℕ} [Fact p.Prime] (hp2
↪  : p ≠ 2)
    (m : ZMod p) (r : {x : ZMod p // x ≠ 0}) :
    squareWitnessMap m r ^ 2 - m = ((r.1 - m / r.1) / 2) ^
     ↪  2 := by
  have h2 : (2 : ZMod p) ≠ 0 := by
    intro hzero
    have hpdvd2 : p ∣ 2 := by
      simpa using (ZMod.natCast_eq_zero_iff 2 p).mp hzero
    have hp_le2 : p ≤ 2 := Nat.le_of_dvd (by decide : 0 <
     ↪  2) hpdvd2
    have hp_ge2 : 2 ≤ p := (Fact.out : Nat.Prime p).two_le
    omega
  unfold squareWitnessMap
  field_simp [r.2, h2]
  ring

lemma squareWitnessMap_isSquare {p : ℕ} [Fact p.Prime]
↪  (hp2 : p ≠ 2)
    (m : ZMod p) (r : {x : ZMod p // x ≠ 0}) :
    IsSquare (squareWitnessMap m r ^ 2 - m) := by
  refine ⟨(r.1 - m / r.1) / 2, by
    simpa [pow_two] using squareWitnessMap_sq_sub hp2 m r⟩

lemma squareWitnessMap_fiber_isRoot {p : ℕ} [Fact p.Prime]
↪  (hp2 : p ≠ 2)
    {m t : ZMod p} {r : {x : ZMod p // x ≠ 0}} (hr :
     ↪  squareWitnessMap m r = t) :
    IsRoot (X ^ 2 - C (2 * t) * X + C m : Polynomial (ZMod
     ↪  p)) r.1 := by
  have h2 : (2 : ZMod p) ≠ 0 := by
    intro hzero
    have hpdvd2 : p ∣ 2 := by
      simpa using (ZMod.natCast_eq_zero_iff 2 p).mp hzero
    have hp_le2 : p ≤ 2 := Nat.le_of_dvd (by decide : 0 <
     ↪  2) hpdvd2
    have hp_ge2 : 2 ≤ p := (Fact.out : Nat.Prime p).two_le
    omega
  have h : (r.1 + m / r.1) / 2 = t := by
    simpa [squareWitnessMap] using hr
  field_simp [r.2, h2] at h
  have hpoly : r.1 ^ 2 - (2 * t) * r.1 + m = 0 := by
    have hzero : r.1 ^ 2 + m - t * (r.1 * 2) = 0 := by
```

```
      exact sub_eq_zero.mpr (by simpa [add_comm,
       ↪  add_left_comm, add_assoc, mul_comm,
       ↪  mul_left_comm,
        mul_assoc] using h)
    simpa [sub_eq_add_neg, add_comm, add_left_comm,
     ↪  add_assoc, mul_comm, mul_left_comm, mul_assoc]
      using hzero
  simpa [Polynomial.IsRoot, eval_add, eval_sub, eval_mul,
   ↪  eval_pow, eval_X, eval_C, pow_two,
    sub_eq_add_neg, add_comm, add_left_comm, add_assoc,
     ↪  mul_comm, mul_left_comm, mul_assoc]
    using hpoly

lemma squareWitnessMap_fiber_card_le_two {p : ℕ} [Fact
 ↪  p.Prime] (hp2 : p ≠ 2)
    (m t : ZMod p) :
    Fintype.card {r : {x : ZMod p // x ≠ 0} //
     ↪  squareWitnessMap m r = t} ≤ 2 := by
  classical
  let poly : Polynomial (ZMod p) := X ^ 2 - C (2 * t) * X +
   ↪  C m
  have hpoly0 : poly ≠ 0 := by
    intro hzero
    have hdeg : poly.natDegree = 2 := by
      rw [show poly = C (1 : ZMod p) * X ^ 2 + C (-(2 * t))
       ↪  * X + C m by
          simp [poly, sub_eq_add_neg, add_assoc,
           ↪  add_left_comm, add_comm]]
      simpa using (Polynomial.natDegree_quadratic (a := (1
       ↪  : ZMod p)) (b := -(2 * t)) (c := m)
        one_ne_zero)
    simpa [hzero] using hdeg
  let g :
      {r : {x : ZMod p // x ≠ 0} // squareWitnessMap m r =
       ↪  t} →
        poly.roots.toFinset := fun r =>
          ⟨r.1.1, by
            rw [Multiset.mem_toFinset,
             ↪  Polynomial.mem_roots]
            · exact squareWitnessMap_fiber_isRoot hp2 r.2
            · exact hpoly0⟩
  have hg : Function.Injective g := by
    intro r s h
    ext
    exact congrArg (fun z : poly.roots.toFinset => (z :
     ↪  ZMod p)) h
  calc
```

```
    Fintype.card {r : {x : ZMod p // x ≠ 0} //
     ↪  squareWitnessMap m r = t}
        ≤ Fintype.card poly.roots.toFinset :=
         ↪  Fintype.card_le_of_injective g hg
    _ = poly.roots.toFinset.card := by rw
     ↪  [Fintype.card_coe]
    _ ≤ poly.roots.card := Multiset.toFinset_card_le
     ↪  poly.roots
    _ ≤ poly.natDegree := Polynomial.card_roots' poly
    _ = 2 := by
      rw [show poly = C (1 : ZMod p) * X ^ 2 + C (-(2 * t))
       ↪  * X + C m by
          simp [poly, sub_eq_add_neg, add_assoc,
           ↪  add_left_comm, add_comm]]
      simpa using (Polynomial.natDegree_quadratic (a := (1
       ↪  : ZMod p)) (b := -(2 * t)) (c := m)
        one_ne_zero)

lemma squareWitnessMap_exists_of_isSquare {p : ℕ} [Fact
 ↪  p.Prime] (hp2 : p ≠ 2)
    {m t : ZMod p} (hm : m ≠ 0) (ht : IsSquare (t ^ 2 - m))
     ↪  :
    ∃ r : {x : ZMod p // x ≠ 0}, squareWitnessMap m r = t
     ↪  := by
  rcases ht with ⟨u, hu⟩
  have h2 : (2 : ZMod p) ≠ 0 := by
    intro hzero
    have hpdvd2 : p ∣ 2 := by
      simpa using (ZMod.natCast_eq_zero_iff 2 p).mp hzero
    have hp_le2 : p ≤ 2 := Nat.le_of_dvd (by decide : 0 <
     ↪  2) hpdvd2
    have hp_ge2 : 2 ≤ p := (Fact.out : Nat.Prime p).two_le
    omega
  have hprod : (t + u) * (t - u) = m := by
    have hprod' : t ^ 2 - u ^ 2 = m := by
      calc
        t ^ 2 - u ^ 2 = t ^ 2 - (u * u) := by simp [pow_two]
         _ = t ^ 2 - (t ^ 2 - m) := by rw [hu]
         _ = m := by ring
    simpa [sq_sub_sq] using hprod'
  have hr : t + u ≠ 0 := by
    intro hzero
    have : m = 0 := by
      rw [← hprod, hzero]
      simp
    exact hm this
  refine ⟨⟨t + u, hr⟩, ?_⟩
```

```
  unfold squareWitnessMap
  apply (div_eq_iff h2).2
  have hmdiv : m / (t + u) = t - u := by
    apply (div_eq_iff hr).2
    calc
      m = (t + u) * (t - u) := hprod.symm
      _ = (t - u) * (t + u) := by ring
  rw [hmdiv]
  ring

lemma card_sq_sub_solution_sigma {p : ℕ} [Fact p.Prime]
 ↪  (hp2 : p ≠ 2)
    {m : ZMod p} (hm : m ≠ 0) :
    Fintype.card (Σ t : ZMod p, {u : ZMod p // u ^ 2 = t ^
     ↪  2 - m}) = p - 1 := by
  classical
  have h2 : (2 : ZMod p) ≠ 0 := by
    intro hzero
    have hpdvd2 : p ∣ 2 := by
      simpa using (ZMod.natCast_eq_zero_iff 2 p).mp hzero
    have hp_le2 : p ≤ 2 := Nat.le_of_dvd (by decide : 0 <
     ↪  2) hpdvd2
    have hp_ge2 : 2 ≤ p := (Fact.out : Nat.Prime p).two_le
    omega
  let S : Type := {tu : ZMod p × ZMod p // tu.2 ^ 2 = tu.1
   ↪  ^ 2 - m}
  let eSigma : (Σ t : ZMod p, {u : ZMod p // u ^ 2 = t ^ 2
   ↪  - m}) ≃ S := {
    toFun := fun tu => ⟨(tu.1, tu.2.1), tu.2.2⟩
    invFun := fun tu => ⟨tu.1.1, ⟨tu.1.2, tu.2⟩⟩
    left_inv := by intro tu; cases tu; rfl
    right_inv := by intro tu; cases tu; rfl
  }
  let e : {r : ZMod p // r ≠ 0} ≃ S := {
    toFun := fun r =>
      ⟨(squareWitnessMap m r, (r.1 - m / r.1) / 2),
       ↪  (squareWitnessMap_sq_sub hp2 m r).symm⟩
    invFun := fun tu => by
      refine ⟨tu.1.1 + tu.1.2, ?_⟩
      intro hzero
      have hneg : tu.1.1 = -tu.1.2 :=
       ↪  eq_neg_of_add_eq_zero_left hzero
      have hsq : tu.1.1 ^ 2 = tu.1.2 ^ 2 := by
        simpa [hneg, pow_two]
      have hm0 : m = 0 := by
        calc
          m = tu.1.1 ^ 2 - tu.1.2 ^ 2 := by rw [tu.2]; ring
          _ = 0 := by rw [hsq]; ring
```

```
      exact hm hm0
    left_inv := by
      intro r
      apply Subtype.ext
      change ((r.1 + m / r.1) / 2 + (r.1 - m / r.1) / 2) =
       ↪  r.1
      field_simp [h2, r.2]
      ring
    right_inv := by
      intro tu
      apply Subtype.ext
      rcases tu with ⟨⟨t, u⟩, hu⟩
      have hsum : t + u ≠ 0 := by
        intro hzero
        have hneg : t = -u := eq_neg_of_add_eq_zero_left
         ↪  hzero
        have hsq : t ^ 2 = u ^ 2 := by
          simpa [hneg, pow_two]
        have hm0 : m = 0 := by
          calc
            m = t ^ 2 - u ^ 2 := by rw [hu]; ring
            _ = 0 := by rw [hsq]; ring
        exact hm hm0
      have hprod : (t + u) * (t - u) = m := by
        calc
          (t + u) * (t - u) = t ^ 2 - u ^ 2 := by ring
          _ = t ^ 2 - (t ^ 2 - m) := by rw [hu]
          _ = m := by ring
      have hmdiv : m / (t + u) = t - u := by
        apply (div_eq_iff hsum).2
        calc
          m = (t + u) * (t - u) := hprod.symm
          _ = (t - u) * (t + u) := by ring
      ext
      · unfold squareWitnessMap
        apply (div_eq_iff h2).2
        rw [hmdiv]
        ring
      · change ((t + u - m / (t + u)) / 2) = u
        rw [hmdiv]
        apply (div_eq_iff h2).2
        ring
  }
  calc
    Fintype.card (Σ t : ZMod p, {u : ZMod p // u ^ 2 = t ^
     ↪  2 - m})
        = Fintype.card S := Fintype.card_congr eSigma
```

```
    _ = Fintype.card {r : ZMod p // r ≠ 0} :=
     ↪  Fintype.card_congr e.symm
    _ = p - 1 := card_nonzero_zmod

lemma quadraticChar_sum_sq_sub {p : ℕ} [Fact p.Prime] (hp2
 ↪  : p ≠ 2)
    {m : ZMod p} (hm : m ≠ 0) :
    ∑ t : ZMod p, quadraticChar (ZMod p) (t ^ 2 - m) = -1
     ↪  := by
  have hchar : ringChar (ZMod p) ≠ 2 := by
    simpa [ZMod.ringChar_zmod_n p] using hp2
  have hrootCard :
      ∀ t : ZMod p,
        ((Fintype.card {u : ZMod p // u ^ 2 = t ^ 2 - m} :
         ↪  ℕ) : ℤ) =
          quadraticChar (ZMod p) (t ^ 2 - m) + 1 := by
    intro t
    calc
      ((Fintype.card {u : ZMod p // u ^ 2 = t ^ 2 - m} : ℕ)
       ↪  : ℤ)
          = ((Fintype.card ↥({x : ZMod p | x ^ 2 = t ^ 2 -
           ↪  m}) : ℕ) : ℤ) := by
              simpa using congrArg (fun n : ℕ => (n : ℤ))
               ↪  (Fintype.card_congr (Equiv.refl _))
      _ = ({x : ZMod p | x ^ 2 = t ^ 2 - m}.toFinset.card :
       ↪  ℤ) := by
            rw [Set.toFinset_card]
      _ = quadraticChar (ZMod p) (t ^ 2 - m) + 1 := by
            simpa using quadraticChar_card_sqrts hchar (t ^
             ↪  2 - m)
  have hsigma :
      ((Fintype.card (∑ t : ZMod p, {u : ZMod p // u ^ 2 =
       ↪  t ^ 2 - m}) : ℕ) : ℤ) =
        ∑ t : ZMod p, (((Fintype.card {u : ZMod p // u ^ 2
         ↪  = t ^ 2 - m} : ℕ) : ℤ)) := by
    rw [Fintype.card_sigma]
    norm_num
  have hcard :
      ((Fintype.card (∑ t : ZMod p, {u : ZMod p // u ^ 2 =
       ↪  t ^ 2 - m}) : ℕ) : ℤ) =
        ((p - 1 : ℕ) : ℤ) := by
    exact_mod_cast card_sq_sub_solution_sigma hp2 hm
  calc
    ∑ t : ZMod p, quadraticChar (ZMod p) (t ^ 2 - m)
        = (∑ t : ZMod p, (quadraticChar (ZMod p) (t ^ 2 -
         ↪  m) + 1)) - p := by
            rw [Finset.sum_add_distrib]
```

```
            simp
    _ = (∑ t : ZMod p, (((Fintype.card {u : ZMod p // u ^ 2
     ↪  = t ^ 2 - m} : ℕ) : ℤ))) - p := by
          apply congrArg (fun z : ℤ => z - p)
          apply Finset.sum_congr rfl
          intro t ht
          symm
          exact hrootCard t
    _ = ((p - 1 : ℕ) : ℤ) - p := by rw [← hsigma, hcard]
    _ = -1 := by
          have hp1 : 1 ≤ p := (Fact.out : Nat.Prime p).pos
          rw [Nat.cast_sub hp1]
          ring

lemma card_roots_sq_sub_eq_of_isSquare {p : ℕ} [Fact
 ↪  p.Prime] (hp2 : p ≠ 2)
    {m : ZMod p} (hm : m ≠ 0) (hsm : IsSquare m) :
    Fintype.card {t : ZMod p // t ^ 2 = m} = 2 := by
  have hchar : ringChar (ZMod p) ≠ 2 := by
    simpa [ZMod.ringChar_zmod_n p] using hp2
  have hcardInt : ((Fintype.card {t : ZMod p // t ^ 2 = m}
   ↪  : ℕ) : ℤ) = 2 := by
    calc
      ((Fintype.card {t : ZMod p // t ^ 2 = m} : ℕ) : ℤ)
          = quadraticChar (ZMod p) m + 1 := by
              calc
                ((Fintype.card {t : ZMod p // t ^ 2 = m} :
                 ↪  ℕ) : ℤ)
                    = ((Fintype.card ↥({x : ZMod p | x ^ 2
                     ↪  = m}) : ℕ) : ℤ) := by
                        simpa using
                          congrArg (fun n : ℕ => (n : ℤ))
                           ↪  (Fintype.card_congr
                           ↪  (Equiv.refl _))
                _ = ({x : ZMod p | x ^ 2 = m}.toFinset.card
                 ↪  : ℤ) := by
                        rw [Set.toFinset_card]
                _ = quadraticChar (ZMod p) m + 1 := by
                      simpa using quadraticChar_card_sqrts
                       ↪  hchar m
      _ = 2 := by
            rw [(quadraticChar_one_iff_isSquare hm).2 hsm]
            norm_num
  omega

lemma card_roots_sq_sub_eq_of_not_isSquare {p : ℕ} [Fact
 ↪  p.Prime] (hp2 : p ≠ 2)
    {m : ZMod p} (hm : m ≠ 0) (hsm : ¬ IsSquare m) :
```

```
    Fintype.card {t : ZMod p // t ^ 2 = m} = 0 := by
  have hchar : ringChar (ZMod p) ≠ 2 := by
    simpa [ZMod.ringChar_zmod_n p] using hp2
  have hcardInt : ((Fintype.card {t : ZMod p // t ^ 2 = m}
   ↪  : ℕ) : ℤ) = 0 := by
    calc
      ((Fintype.card {t : ZMod p // t ^ 2 = m} : ℕ) : ℤ)
          = quadraticChar (ZMod p) m + 1 := by
              calc
                ((Fintype.card {t : ZMod p // t ^ 2 = m} :
                 ↪  ℕ) : ℤ)
                    = ((Fintype.card ↥({x : ZMod p | x ^ 2
                     ↪  = m}) : ℕ) : ℤ) := by
                        simpa using
                          congrArg (fun n : ℕ => (n : ℤ))
                           ↪  (Fintype.card_congr
                           ↪  (Equiv.refl _))
                _ = ({x : ZMod p | x ^ 2 = m}.toFinset.card
                 ↪  : ℤ) := by
                        rw [Set.toFinset_card]
                _ = quadraticChar (ZMod p) m + 1 := by
                      simpa using quadraticChar_card_sqrts
                       ↪  hchar m
      _ = 0 := by
            rw [(quadraticChar_neg_one_iff_not_isSquare (F
             ↪  := ZMod p)).2 hsm]
            norm_num
  omega

lemma quadraticChar_sq_sub_eq {p : ℕ} [Fact p.Prime] (hp2
 ↪  : p ≠ 2)
    {m t : ZMod p} :
    quadraticChar (ZMod p) (t ^ 2 - m) =
      1 - (if t ^ 2 = m then 1 else 0) -
        2 * (if ¬ IsSquare (t ^ 2 - m) then 1 else 0) := by
  by_cases hsq : IsSquare (t ^ 2 - m)
  · by_cases hzero : t ^ 2 = m
    · have hchar0 : quadraticChar (ZMod p) (t ^ 2 - m) = 0
     ↪  := by
        simp [sub_eq_zero.mpr hzero]
      simp [hzero, hsq]
    · have hne : t ^ 2 - m ≠ 0 := by
        simpa [sub_eq_zero] using hzero
      have hchar1 : quadraticChar (ZMod p) (t ^ 2 - m) = 1
       ↪  := by
        exact (quadraticChar_one_iff_isSquare hne).2 hsq
      simp [hzero, hsq, hchar1]
```

```
  · have hcharNeg : quadraticChar (ZMod p) (t ^ 2 - m) = -1
   ↪  := by
      exact (quadraticChar_neg_one_iff_not_isSquare (F :=
       ↪  ZMod p)).2 hsq
    have hzero : t ^ 2 ≠ m := by
      intro ht
      apply hsq
      refine ⟨0, ?_⟩
      simpa [sub_eq_zero.mpr ht]
    simp [hsq, hzero, hcharNeg]

lemma card_nonsquare_sq_sub_eq_of_isSquare {p : ℕ} [Fact
 ↪  p.Prime] (hp2 : p ≠ 2)
    {m : ZMod p} (hm : m ≠ 0) (hsm : IsSquare m) :
    Fintype.card {t : ZMod p // ¬ IsSquare (t ^ 2 - m)} =
     ↪  (p - 1) / 2 := by
  classical
  let N : ℕ := Fintype.card {t : ZMod p // ¬ IsSquare (t ^
   ↪  2 - m)}
  let Z : ℕ := Fintype.card {t : ZMod p // t ^ 2 = m}
  have hsum0 :
      ∑ t : ZMod p, quadraticChar (ZMod p) (t ^ 2 - m) =
        ((p : ℕ) : ℤ) - Z - 2 * N := by
    calc
      ∑ t : ZMod p, quadraticChar (ZMod p) (t ^ 2 - m)
          = ∑ t : ZMod p,
              (1 - (if t ^ 2 = m then 1 else 0) -
                2 * (if ¬ IsSquare (t ^ 2 - m) then 1 else
                 ↪  0) : ℤ) := by
              apply Finset.sum_congr rfl
              intro t ht
              simpa using quadraticChar_sq_sub_eq (p := p)
               ↪  hp2 (m := m) (t := t)
      _ = (∑ _t : ZMod p, (1 : ℤ)) -
            (∑ t : ZMod p, (if t ^ 2 = m then 1 else 0 : ℤ))
             ↪  -
            (∑ t : ZMod p, (2 * (if ¬ IsSquare (t ^ 2 - m)
             ↪  then 1 else 0 : ℤ))) := by
              rw [Finset.sum_sub_distrib,
               ↪  Finset.sum_sub_distrib]
      _ = (∑ _t : ZMod p, (1 : ℤ)) -
            (∑ t : ZMod p, (if t ^ 2 = m then 1 else 0 : ℤ))
             ↪  -
            2 * (∑ t : ZMod p, (if ¬ IsSquare (t ^ 2 - m)
             ↪  then 1 else 0 : ℤ)) := by
              rw [← Finset.mul_sum]
      _ = ((p : ℕ) : ℤ) - Z - 2 * N := by
```

```
            have hsum1 : ∑ _t : ZMod p, (1 : ℤ) = p := by
             ↪  simp [ZMod.card]
            have hsumZ : ∑ t : ZMod p, (if t ^ 2 = m then 1
             ↪  else 0 : ℤ) = Z := by
              simpa [Z, Fintype.card_subtype] using
                (Finset.sum_boole (fun t : ZMod p => t ^ 2
                 ↪  = m)
                  (Finset.univ : Finset (ZMod p)) : _)
            have hsumN : ∑ t : ZMod p, (if ¬ IsSquare (t ^
             ↪  2 - m) then 1 else 0 : ℤ) = N := by
              simpa [N, Fintype.card_subtype] using
                (Finset.sum_boole (fun t : ZMod p => ¬
                 ↪  IsSquare (t ^ 2 - m))
                  (Finset.univ : Finset (ZMod p)) : _)
            rw [hsum1, hsumZ, hsumN]
  have hZ : Z = 2 := card_roots_sq_sub_eq_of_isSquare hp2
   ↪  hm hsm
  have hNint : ((N : ℕ) : ℤ) = (((p - 1) / 2 : ℕ) : ℤ) := by
    have hsum1 : ∑ t : ZMod p, quadraticChar (ZMod p) (t ^
     ↪  2 - m) = -1 :=
      quadraticChar_sum_sq_sub hp2 hm
    rw [hZ] at hsum0
    omega
  exact_mod_cast hNint

lemma card_nonsquare_sq_sub_eq_of_not_isSquare {p : ℕ}
 ↪  [Fact p.Prime] (hp2 : p ≠ 2)
    {m : ZMod p} (hm : m ≠ 0) (hsm : ¬ IsSquare m) :
    Fintype.card {t : ZMod p // ¬ IsSquare (t ^ 2 - m)} =
     ↪  (p + 1) / 2 := by
  classical
  let N : ℕ := Fintype.card {t : ZMod p // ¬ IsSquare (t ^
   ↪  2 - m)}
  let Z : ℕ := Fintype.card {t : ZMod p // t ^ 2 = m}
  have hsum0 :
      ∑ t : ZMod p, quadraticChar (ZMod p) (t ^ 2 - m) =
        ((p : ℕ) : ℤ) - Z - 2 * N := by
    calc
      ∑ t : ZMod p, quadraticChar (ZMod p) (t ^ 2 - m)
          = ∑ t : ZMod p,
              (1 - (if t ^ 2 = m then 1 else 0) -
                2 * (if ¬ IsSquare (t ^ 2 - m) then 1 else
                 ↪  0) : ℤ) := by
              apply Finset.sum_congr rfl
              intro t ht
              simpa using quadraticChar_sq_sub_eq (p := p)
               ↪  hp2 (m := m) (t := t)
```

```
      _ = (∑ _t : ZMod p, (1 : ℤ)) -
            (∑ t : ZMod p, (if t ^ 2 = m then 1 else 0 : ℤ))
             ↪  -
            (∑ t : ZMod p, (2 * (if ¬ IsSquare (t ^ 2 - m)
             ↪  then 1 else 0 : ℤ))) := by
              rw [Finset.sum_sub_distrib,
               ↪  Finset.sum_sub_distrib]
      _ = (∑ _t : ZMod p, (1 : ℤ)) -
            (∑ t : ZMod p, (if t ^ 2 = m then 1 else 0 : ℤ))
             ↪  -
            2 * (∑ t : ZMod p, (if ¬ IsSquare (t ^ 2 - m)
             ↪  then 1 else 0 : ℤ)) := by
              rw [← Finset.mul_sum]
      _ = ((p : ℕ) : ℤ) - Z - 2 * N := by
            have hsum1 : ∑ _t : ZMod p, (1 : ℤ) = p := by
             ↪  simp [ZMod.card]
            have hsumZ : ∑ t : ZMod p, (if t ^ 2 = m then 1
             ↪  else 0 : ℤ) = Z := by
              simpa [Z, Fintype.card_subtype] using
                (Finset.sum_boole (fun t : ZMod p => t ^ 2
                 ↪  = m)
                  (Finset.univ : Finset (ZMod p)) : _)
            have hsumN : ∑ t : ZMod p, (if ¬ IsSquare (t ^
             ↪  2 - m) then 1 else 0 : ℤ) = N := by
              simpa [N, Fintype.card_subtype] using
                (Finset.sum_boole (fun t : ZMod p => ¬
                 ↪  IsSquare (t ^ 2 - m))
                  (Finset.univ : Finset (ZMod p)) : _)
            rw [hsum1, hsumZ, hsumN]
  have hZ : Z = 0 := card_roots_sq_sub_eq_of_not_isSquare
   ↪  hp2 hm hsm
  have hNint : ((N : ℕ) : ℤ) = (((p + 1) / 2 : ℕ) : ℤ) := by
    have hsum1 : ∑ t : ZMod p, quadraticChar (ZMod p) (t ^
     ↪  2 - m) = -1 :=
      quadraticChar_sum_sq_sub hp2 hm
    rw [hZ] at hsum0
    omega
  exact_mod_cast hNint

lemma card_nonsquare_sq_sub_le {p : ℕ} [Fact p.Prime] (hp2
 ↪  : p ≠ 2)
    {m : ZMod p} (hm : m ≠ 0) :
    Fintype.card {t : ZMod p // ¬IsSquare (t ^ 2 - m)} ≤ (p
     ↪  + 1) / 2 := by
  classical
  let s : Finset (ZMod p) := Finset.univ.image
   ↪  (squareWitnessMap m)
```

```
have hs_bound : p - 1 ≤ 2 * s.card := by
  have hs :=
    Finset.card_le_mul_card_image (s := (Finset.univ :
     ↪  Finset {x : ZMod p // x ≠ 0})) 2
      (fun t ht => by
        have hfilter :
           (Finset.univ.filter (fun a : {x : ZMod p // x ≠
             ↪  0} => squareWitnessMap m a = t)).card =
              Fintype.card {r : {x : ZMod p // x ≠ 0} //
               ↪  squareWitnessMap m r = t} := by
          let e :
              (Finset.univ.filter (fun a : {x : ZMod p //
               ↪  x ≠ 0} => squareWitnessMap m a = t)) ≃
                {r : {x : ZMod p // x ≠ 0} //
                 ↪  squareWitnessMap m r = t} := {
            toFun := fun a => by
              rcases a with ⟨a, ha⟩
              have hat : squareWitnessMap m a = t := by
                simpa using ha
              exact ⟨a, hat⟩
            invFun := fun r => by
              refine ⟨r.1, ?_⟩
              simp [r.2]
            left_inv := by intro a; ext; rfl
            right_inv := by intro r; ext; rfl
          }
          calc
            (Finset.univ.filter (fun a : {x : ZMod p // x
             ↪  ≠ 0} => squareWitnessMap m a = t)).card
                = Fintype.card
                    (Finset.univ.filter (fun a : {x :
                     ↪  ZMod p // x ≠ 0} =>
                     ↪  squareWitnessMap m a = t)) := by
                      symm
                      exact Fintype.card_coe _
            _ = Fintype.card {r : {x : ZMod p // x ≠ 0}
             ↪  // squareWitnessMap m r = t} :=
                  Fintype.card_congr e
        rw [hfilter]
        exact
          (squareWitnessMap_fiber_card_le_two (p := p)
           ↪  hp2 m t :
            Fintype.card {r : {x : ZMod p // x ≠ 0} //
             ↪  squareWitnessMap m r = t} ≤ 2))
  simpa [s, card_nonzero_zmod (p := p), mul_comm] using
   ↪  hs
have hs_lower : (p - 1) / 2 ≤ s.card := by
```

```
  obtain ⟨k, hk⟩ := ((Fact.out : Nat.Prime
   ↪  p).even_sub_one hp2).two_dvd
  rw [hk] at hs_bound ⊢
  omega
have hs_square :
    s.card ≤ Fintype.card {t : ZMod p // IsSquare (t ^ 2
     ↪  - m)} := by
  let g : s → {t : ZMod p // IsSquare (t ^ 2 - m)} := fun
   ↪  t =>
    ⟨t.1, by
      rcases Finset.mem_image.mp t.2 with ⟨r, -, hr⟩
      rw [← hr]
      exact squareWitnessMap_isSquare hp2 m r⟩
  simpa using
    (Fintype.card_le_of_injective g (fun _ _ h =>
      Subtype.ext (congrArg (fun z : {t : ZMod p //
       ↪  IsSquare (t ^ 2 - m)} => (z : ZMod p)) h)))
have hsquare_lower : (p - 1) / 2 ≤ Fintype.card {t : ZMod
 ↪  p // IsSquare (t ^ 2 - m)} :=
  hs_lower.trans hs_square
have hcompl :
    Fintype.card {t : ZMod p // ¬IsSquare (t ^ 2 - m)} =
      Fintype.card (ZMod p) - Fintype.card {t : ZMod p //
       ↪  IsSquare (t ^ 2 - m)} := by
  simpa using
    (Fintype.card_subtype_compl (fun t : ZMod p =>
     ↪  IsSquare (t ^ 2 - m)) :
     Fintype.card {t : ZMod p // ¬IsSquare (t ^ 2 - m)} =
        Fintype.card (ZMod p) - Fintype.card {t : ZMod p
         ↪  // IsSquare (t ^ 2 - m)})
let squareCard : ℕ := Fintype.card {t : ZMod p //
 ↪  IsSquare (t ^ 2 - m)}
have hcard : Fintype.card (ZMod p) = p := ZMod.card p
obtain ⟨k, hk⟩ := ((Fact.out : Nat.Prime p).even_sub_one
 ↪  hp2).two_dvd
have hp_ge1 : 1 ≤ p := (Fact.out : Nat.Prime p).pos
have hpodd : p = 2 * k + 1 := by
  omega
have hkhalf : (p - 1) / 2 = k := by
  omega
have hsquare_lower' : k ≤ squareCard := by
  simpa [hkhalf] using hsquare_lower
have hnonsq : Fintype.card {t : ZMod p // ¬IsSquare (t ^
 ↪  2 - m)} = p - squareCard := by
  rw [hcompl, hcard]
rw [hnonsq]
omega
```

```
lemma quadCoeff_eq_of_eval_eq_three {p : ℕ} [Fact p.Prime]
 ↪  {v w : QuadCoeff p}
    {x₁ x₂ x₃ : ZMod p} (hx₁₂ : x₁ ≠ x₂) (hx₁₃ : x₁ ≠ x₃)
     ↪  (hx₂₃ : x₂ ≠ x₃)
    (h₁ : quadCoeffEval v x₁ = quadCoeffEval w x₁)
    (h₂ : quadCoeffEval v x₂ = quadCoeffEval w x₂)
    (h₃ : quadCoeffEval v x₃ = quadCoeffEval w x₃) : v = w
     ↪  := by
  let d : QuadCoeff p := fun i => v i - w i
  have hd₁ : quadCoeffEval d x₁ = 0 := by
    have hzero : quadCoeffEval v x₁ - quadCoeffEval w x₁ =
     ↪  0 := by simpa [h₁]
    simpa [d, quadCoeffEval, Fin.sum_univ_three,
     ↪  sub_eq_add_neg, add_comm, add_left_comm,
      add_assoc, mul_add, add_mul] using hzero
  have hd₂ : quadCoeffEval d x₂ = 0 := by
    have hzero : quadCoeffEval v x₂ - quadCoeffEval w x₂ =
     ↪  0 := by simpa [h₂]
    simpa [d, quadCoeffEval, Fin.sum_univ_three,
     ↪  sub_eq_add_neg, add_comm, add_left_comm,
      add_assoc, mul_add, add_mul] using hzero
  have hd₃ : quadCoeffEval d x₃ = 0 := by
    have hzero : quadCoeffEval v x₃ - quadCoeffEval w x₃ =
     ↪  0 := by simpa [h₃]
    simpa [d, quadCoeffEval, Fin.sum_univ_three,
     ↪  sub_eq_add_neg, add_comm, add_left_comm,
      add_assoc, mul_add, add_mul] using hzero
  have hfactor₁₂ :
      quadCoeffEval d x₂ - quadCoeffEval d x₁ =
        (x₂ - x₁) * (d 2 * (x₂ + x₁) + d 1) := by
    simp [quadCoeffEval, d, Fin.sum_univ_three, pow_two]
    ring
  have hfactor₁₃ :
      quadCoeffEval d x₃ - quadCoeffEval d x₁ =
        (x₃ - x₁) * (d 2 * (x₃ + x₁) + d 1) := by
    simp [quadCoeffEval, d, Fin.sum_univ_three, pow_two]
    ring
  have hlin₁₂ : (x₂ - x₁) * (d 2 * (x₂ + x₁) + d 1) = 0 :=
   ↪  by
    rw [← hfactor₁₂, hd₂, hd₁]
    simp
  have hlin₁₃ : (x₃ - x₁) * (d 2 * (x₃ + x₁) + d 1) = 0 :=
   ↪  by
    rw [← hfactor₁₃, hd₃, hd₁]
    simp
  have haux₁₂ : d 2 * (x₂ + x₁) + d 1 = 0 := by
```

```
    exact (mul_eq_zero.mp hlin₁₂).resolve_left
     ↪ (sub_ne_zero.mpr hx₁₂.symm)
  have haux₁₃ : d 2 * (x₃ + x₁) + d 1 = 0 := by
    exact (mul_eq_zero.mp hlin₁₃).resolve_left
     ↪ (sub_ne_zero.mpr hx₁₃.symm)
  have hd₂zero : d 2 = 0 := by
    have hsub :
        d 2 * (x₃ - x₂) =
          (d 2 * (x₃ + x₁) + d 1) - (d 2 * (x₂ + x₁) + d 1)
           ↪ := by
      ring
    have : d 2 * (x₃ - x₂) = 0 := by
      rw [hsub, haux₁₃, haux₁₂]
      simp
    exact (mul_eq_zero.mp this).resolve_right
     ↪ (sub_ne_zero.mpr hx₂₃.symm)
  have hd₁zero : d 1 = 0 := by
    simpa [hd₂zero] using haux₁₂
  have hd₀zero : d 0 = 0 := by
    simpa [quadCoeffEval, d, Fin.sum_univ_three, hd₂zero,
     ↪ hd₁zero] using hd₁
  ext i
  fin_cases i
  · simpa [d, sub_eq_zero] using hd₀zero
  · simpa [d, sub_eq_zero] using hd₁zero
  · simpa [d, sub_eq_zero] using hd₂zero

lemma card_rootlessQuad_eval_pair_le {p : ℕ} [Fact
 ↪ p.Prime] (hp2 : p ≠ 2)
    {x₁ x₂ y₁ y₂ : ZMod p} (hx : x₁ ≠ x₂) (hy₁ : y₁ ≠ 0)
     ↪ (hy₂ : y₂ ≠ 0) :
    Fintype.card {q : RootlessQuad p //
        quadCoeffEval q.1 x₁ = y₁ ∧ quadCoeffEval q.1 x₂ =
         ↪ y₂} ≤ (p + 1) / 2 := by
  classical
  let d : ZMod p := x₂ - x₁
  have hd : d ≠ 0 := sub_ne_zero.mpr hx.symm
  have hd2 : d ^ 2 ≠ 0 := pow_ne_zero 2 hd
  have hm : (4 * y₁ * y₂ : ZMod p) ≠ 0 := by
    have h4 : (4 : ZMod p) ≠ 0 := by
      intro hzero
      have hpdvd4 : p ∣ 4 := by
        simpa using (ZMod.natCast_eq_zero_iff 4 p).mp hzero
      have hpdvd2pow : p ∣ 2 ^ 2 := by simpa [pow_two]
       ↪ using hpdvd4
      have hpdvd2 : p ∣ 2 := (Fact.out : Nat.Prime
       ↪ p).dvd_of_dvd_pow hpdvd2pow
```

```
      have hp_le2 : p ≤ 2 := Nat.le_of_dvd (by decide : 0 <
       ↪  2) hpdvd2
      have hp_ge2 : 2 ≤ p := (Fact.out : Nat.Prime p).two_le
       omega
    have h4y : (4 * y₁ : ZMod p) ≠ 0 := mul_ne_zero h4 hy₁
    exact mul_ne_zero h4y hy₂
  let g :
      {q : RootlessQuad p // quadCoeffEval q.1 x₁ = y₁ ∧
       ↪  quadCoeffEval q.1 x₂ = y₂} →
        {t : ZMod p // ¬IsSquare (t ^ 2 - 4 * y₁ * y₂)} :=
         ↪  fun q =>
          ⟨y₁ + y₂ - d ^ 2 * q.1.1 2, by
            intro hs
            apply q.1.2.2
            rcases hs with ⟨s, hs⟩
            refine ⟨s / d, ?_⟩
            have hdisc0 :
                d ^ 2 * quadDiscr q.1.1 =
                  (quadCoeffEval q.1.1 x₁ + quadCoeffEval
                   ↪  q.1.1 x₂ - d ^ 2 * q.1.1 2) ^ 2 -
                    4 * quadCoeffEval q.1.1 x₁ *
                     ↪  quadCoeffEval q.1.1 x₂ := by
              simp [quadDiscr, quadCoeffEval, d,
               ↪  Fin.sum_univ_three, pow_two]
              ring
            have hdisc :
                d ^ 2 * quadDiscr q.1.1 =
                  (y₁ + y₂ - d ^ 2 * q.1.1 2) ^ 2 - 4 * y₁
                   ↪  * y₂ := by
              simpa [q.2.1, q.2.2] using hdisc0
            have hsq : quadDiscr q.1.1 = (s / d) ^ 2 := by
             have hmul : d ^ 2 * quadDiscr q.1.1 = d ^ 2 *
              ↪  ((s / d) ^ 2) := by
                calc
                  d ^ 2 * quadDiscr q.1.1 =
                      (y₁ + y₂ - d ^ 2 * q.1.1 2) ^ 2 - 4 *
                       ↪  y₁ * y₂ := hdisc
                  _ = s * s := hs
                  _ = d ^ 2 * ((s / d) ^ 2) := by
                    field_simp [hd]
              exact (mul_right_injective₀ hd2) (by simpa
               ↪  [mul_comm] using hmul)
            simpa [pow_two] using hsq⟩
  have hg : Function.Injective g := by
    intro q r h
    apply Subtype.ext
    apply Subtype.ext
```

```
have ht : y₁ + y₂ - d ^ 2 * q.1.1 2 = y₁ + y₂ - d ^ 2 *
 ↪  r.1.1 2 := by
  exact congrArg (fun z : {t : ZMod p // ¬IsSquare (t ^
   ↪  2 - 4 * y₁ * y₂)} => (z : ZMod p)) h
have haMul : d ^ 2 * q.1.1 2 = d ^ 2 * r.1.1 2 := by
  calc
    d ^ 2 * q.1.1 2 = (y₁ + y₂) - (y₁ + y₂ - d ^ 2 *
     ↪  q.1.1 2) := by ring
    _ = (y₁ + y₂) - (y₁ + y₂ - d ^ 2 * r.1.1 2) := by
     ↪  rw [ht]
    _ = d ^ 2 * r.1.1 2 := by ring
have ha : q.1.1 2 = r.1.1 2 := by
  exact (mul_right_injective₀ hd2) (by simpa
   ↪  [mul_comm] using haMul)
have hbq0 :
    q.1.1 1 * d =
      -(d * q.1.1 2 * x₁) - d * q.1.1 2 * x₂ +
        quadCoeffEval q.1.1 x₂ - quadCoeffEval q.1.1 x₁
         ↪  := by
  simp [quadCoeffEval, d, Fin.sum_univ_three, pow_two]
  ring
have hbq : q.1.1 1 * d = -(d * q.1.1 2 * x₁) - d *
 ↪  q.1.1 2 * x₂ + y₂ - y₁ := by
  simpa [q.2.1, q.2.2] using hbq0
have hbr0 :
    r.1.1 1 * d =
      -(d * r.1.1 2 * x₁) - d * r.1.1 2 * x₂ +
        quadCoeffEval r.1.1 x₂ - quadCoeffEval r.1.1 x₁
         ↪  := by
  simp [quadCoeffEval, d, Fin.sum_univ_three, pow_two]
  ring
have hbr : r.1.1 1 * d = -(d * r.1.1 2 * x₁) - d *
 ↪  r.1.1 2 * x₂ + y₂ - y₁ := by
  simpa [r.2.1, r.2.2] using hbr0
have hbMul : q.1.1 1 * d = r.1.1 1 * d := by
  calc
    q.1.1 1 * d = -(d * q.1.1 2 * x₁) - d * q.1.1 2 *
     ↪  x₂ + y₂ - y₁ := hbq
    _ = -(d * r.1.1 2 * x₁) - d * r.1.1 2 * x₂ + y₂ -
     ↪  y₁ := by simp [ha]
    _ = r.1.1 1 * d := by simpa using hbr.symm
have hb : q.1.1 1 = r.1.1 1 := by
  exact (mul_left_injective₀ hd) (by simpa [mul_comm]
   ↪  using hbMul)
have hcq0 : q.1.1 0 = quadCoeffEval q.1.1 x₁ - q.1.1 1
 ↪  * x₁ - q.1.1 2 * x₁ ^ 2 := by
  simp [quadCoeffEval, Fin.sum_univ_three]
```

```
      ring
    have hcq : q.1.1 0 = y₁ - q.1.1 1 * x₁ - q.1.1 2 * x₁ ^
     ↪  2 := by
      simpa [q.2.1] using hcq0
    have hcr0 : r.1.1 0 = quadCoeffEval r.1.1 x₁ - r.1.1 1
     ↪  * x₁ - r.1.1 2 * x₁ ^ 2 := by
      simp [quadCoeffEval, Fin.sum_univ_three]
      ring
    have hcr : r.1.1 0 = y₁ - r.1.1 1 * x₁ - r.1.1 2 * x₁ ^
     ↪  2 := by
      simpa [r.2.1] using hcr0
    have hc : q.1.1 0 = r.1.1 0 := by
      calc
       q.1.1 0 = y₁ - q.1.1 1 * x₁ - q.1.1 2 * x₁ ^ 2 := hcq
        _ = y₁ - r.1.1 1 * x₁ - r.1.1 2 * x₁ ^ 2 := by simp
         ↪  [ha, hb]
        _ = r.1.1 0 := by simpa using hcr.symm
    ext i
    fin_cases i <;> assumption
  exact (Fintype.card_le_of_injective g hg).trans
   ↪  (card_nonsquare_sq_sub_le hp2 hm)

lemma card_rootlessQuad_eval_pair_le_square {p : ℕ} [Fact
 ↪  p.Prime] (hp2 : p ≠ 2)
    {x₁ x₂ y₁ y₂ : ZMod p} (hx : x₁ ≠ x₂) (hy₁ : y₁ ≠ 0)
     ↪  (hy₂ : y₂ ≠ 0)
    (hsq : IsSquare (y₁ * y₂)) :
    Fintype.card {q : RootlessQuad p //
        quadCoeffEval q.1 x₁ = y₁ ∧ quadCoeffEval q.1 x₂ =
         ↪  y₂} ≤ (p - 1) / 2 := by
  classical
  let d : ZMod p := x₂ - x₁
  have hd : d ≠ 0 := sub_ne_zero.mpr hx.symm
  have hd2 : d ^ 2 ≠ 0 := pow_ne_zero 2 hd
  have hm : (4 * y₁ * y₂ : ZMod p) ≠ 0 := by
    have h4 : (4 : ZMod p) ≠ 0 := by
      intro hzero
      have hpdvd4 : p ∣ 4 := by
        simpa using (ZMod.natCast_eq_zero_iff 4 p).mp hzero
      have hpdvd2pow : p ∣ 2 ^ 2 := by simpa [pow_two]
       ↪  using hpdvd4
      have hpdvd2 : p ∣ 2 := (Fact.out : Nat.Prime
       ↪  p).dvd_of_dvd_pow hpdvd2pow
      have hp_le2 : p ≤ 2 := Nat.le_of_dvd (by decide : 0 <
       ↪  2) hpdvd2
      have hp_ge2 : 2 ≤ p := (Fact.out : Nat.Prime p).two_le
      omega
```

```
  have h4y : (4 * y₁ : ZMod p) ≠ 0 := mul_ne_zero h4 hy₁
  exact mul_ne_zero h4y hy₂
let g :
    {q : RootlessQuad p // quadCoeffEval q.1 x₁ = y₁ ∧
     ↪  quadCoeffEval q.1 x₂ = y₂} →
      {t : ZMod p // ¬IsSquare (t ^ 2 - 4 * y₁ * y₂)} :=
       ↪  fun q =>
        ⟨y₁ + y₂ - d ^ 2 * q.1.1 2, by
          intro hs
          apply q.1.2.2
          rcases hs with ⟨s, hs⟩
          refine ⟨s / d, ?_⟩
          have hdisc0 :
              d ^ 2 * quadDiscr q.1.1 =
                (quadCoeffEval q.1.1 x₁ + quadCoeffEval
                 ↪  q.1.1 x₂ - d ^ 2 * q.1.1 2) ^ 2 -
                  4 * quadCoeffEval q.1.1 x₁ *
                   ↪  quadCoeffEval q.1.1 x₂ := by
            simp [quadDiscr, quadCoeffEval, d,
             ↪  Fin.sum_univ_three, pow_two]
            ring
          have hdisc :
              d ^ 2 * quadDiscr q.1.1 =
                (y₁ + y₂ - d ^ 2 * q.1.1 2) ^ 2 - 4 * y₁
                 ↪  * y₂ := by
            simpa [q.2.1, q.2.2] using hdisc0
          have hsq : quadDiscr q.1.1 = (s / d) ^ 2 := by
           have hmul : d ^ 2 * quadDiscr q.1.1 = d ^ 2 *
            ↪  ((s / d) ^ 2) := by
              calc
                d ^ 2 * quadDiscr q.1.1 =
                    (y₁ + y₂ - d ^ 2 * q.1.1 2) ^ 2 - 4 *
                     ↪  y₁ * y₂ := hdisc
                _ = s * s := hs
                _ = d ^ 2 * ((s / d) ^ 2) := by
                  field_simp [hd]
            exact (mul_right_injective₀ hd2) (by simpa
             ↪  [mul_comm] using hmul)
          simpa [pow_two] using hsq⟩
have hg : Function.Injective g := by
  intro q r h
  apply Subtype.ext
  apply Subtype.ext
  have ht : y₁ + y₂ - d ^ 2 * q.1.1 2 = y₁ + y₂ - d ^ 2 *
   ↪  r.1.1 2 := by
    exact congrArg (fun z : {t : ZMod p // ¬IsSquare (t ^
     ↪  2 - 4 * y₁ * y₂)} => (z : ZMod p)) h
```

```
have haMul : d ^ 2 * q.1.1 2 = d ^ 2 * r.1.1 2 := by
  calc
    d ^ 2 * q.1.1 2 = (y₁ + y₂) - (y₁ + y₂ - d ^ 2 *
     ↪  q.1.1 2) := by ring
    _ = (y₁ + y₂) - (y₁ + y₂ - d ^ 2 * r.1.1 2) := by
     ↪  rw [ht]
    _ = d ^ 2 * r.1.1 2 := by ring
have ha : q.1.1 2 = r.1.1 2 := by
  exact (mul_right_injective₀ hd2) (by simpa
   ↪  [mul_comm] using haMul)
have hbq0 :
    q.1.1 1 * d =
      -(d * q.1.1 2 * x₁) - d * q.1.1 2 * x₂ +
       quadCoeffEval q.1.1 x₂ - quadCoeffEval q.1.1 x₁
        ↪  := by
  simp [quadCoeffEval, d, Fin.sum_univ_three, pow_two]
  ring
have hbq : q.1.1 1 * d = -(d * q.1.1 2 * x₁) - d *
 ↪  q.1.1 2 * x₂ + y₂ - y₁ := by
  simpa [q.2.1, q.2.2] using hbq0
have hbr0 :
    r.1.1 1 * d =
      -(d * r.1.1 2 * x₁) - d * r.1.1 2 * x₂ +
       quadCoeffEval r.1.1 x₂ - quadCoeffEval r.1.1 x₁
        ↪  := by
  simp [quadCoeffEval, d, Fin.sum_univ_three, pow_two]
  ring
have hbr : r.1.1 1 * d = -(d * r.1.1 2 * x₁) - d *
 ↪  r.1.1 2 * x₂ + y₂ - y₁ := by
  simpa [r.2.1, r.2.2] using hbr0
have hbMul : q.1.1 1 * d = r.1.1 1 * d := by
  calc
    q.1.1 1 * d = -(d * q.1.1 2 * x₁) - d * q.1.1 2 *
     ↪  x₂ + y₂ - y₁ := hbq
    _ = -(d * r.1.1 2 * x₁) - d * r.1.1 2 * x₂ + y₂ -
     ↪  y₁ := by simp [ha]
    _ = r.1.1 1 * d := by simpa using hbr.symm
have hb : q.1.1 1 = r.1.1 1 := by
  exact (mul_left_injective₀ hd) (by simpa [mul_comm]
   ↪  using hbMul)
have hcq0 : q.1.1 0 = quadCoeffEval q.1.1 x₁ - q.1.1 1
 ↪  * x₁ - q.1.1 2 * x₁ ^ 2 := by
  simp [quadCoeffEval, Fin.sum_univ_three]
  ring
have hcq : q.1.1 0 = y₁ - q.1.1 1 * x₁ - q.1.1 2 * x₁ ^
 ↪  2 := by
  simpa [q.2.1] using hcq0
```

```
    have hcr0 : r.1.1 0 = quadCoeffEval r.1.1 x₁ - r.1.1 1
     ↪  * x₁ - r.1.1 2 * x₁ ^ 2 := by
      simp [quadCoeffEval, Fin.sum_univ_three]
      ring
    have hcr : r.1.1 0 = y₁ - r.1.1 1 * x₁ - r.1.1 2 * x₁ ^
     ↪  2 := by
      simpa [r.2.1] using hcr0
    have hc : q.1.1 0 = r.1.1 0 := by
      calc
       q.1.1 0 = y₁ - q.1.1 1 * x₁ - q.1.1 2 * x₁ ^ 2 := hcq
        _ = y₁ - r.1.1 1 * x₁ - r.1.1 2 * x₁ ^ 2 := by simp
         ↪  [ha, hb]
        _ = r.1.1 0 := by simpa using hcr.symm
    ext i
    fin_cases i <;> assumption
  have hsq4 : IsSquare (4 * y₁ * y₂ : ZMod p) := by
    rcases hsq with ⟨s, hs⟩
    refine ⟨2 * s, by
      calc
        4 * y₁ * y₂ = 4 * (y₁ * y₂) := by ring
        _ = 4 * (s * s) := by rw [hs]
        _ = (2 * s) * (2 * s) := by ring⟩
  have hcard :
      Fintype.card {t : ZMod p // ¬IsSquare (t ^ 2 - 4 * y₁
       ↪  * y₂)} = (p - 1) / 2 := by
    simpa [mul_assoc] using
     ↪  card_nonsquare_sq_sub_eq_of_isSquare (p := p) hp2
     ↪  hm hsq4
  rw [← hcard]
  exact Fintype.card_le_of_injective g hg

lemma card_rootlessQuad_eval_triple_le_one {p : ℕ} [Fact
 ↪  p.Prime]
    {x₁ x₂ x₃ y₁ y₂ y₃ : ZMod p}
    (hx₁₂ : x₁ ≠ x₂) (hx₁₃ : x₁ ≠ x₃) (hx₂₃ : x₂ ≠ x₃) :
    Fintype.card {q : RootlessQuad p //
        quadCoeffEval q.1 x₁ = y₁ ∧
          quadCoeffEval q.1 x₂ = y₂ ∧
          quadCoeffEval q.1 x₃ = y₃} ≤ 1 := by
  classical
  have hsub :
      Subsingleton {q : RootlessQuad p //
          quadCoeffEval q.1 x₁ = y₁ ∧
            quadCoeffEval q.1 x₂ = y₂ ∧
            quadCoeffEval q.1 x₃ = y₃} := by
    refine ⟨?_⟩
    intro q r
    apply Subtype.ext
```

```
    apply Subtype.ext
    exact quadCoeff_eq_of_eval_eq_three hx₁₂ hx₁₃ hx₂₃
      (q.2.1.trans r.2.1.symm)
      (q.2.2.1.trans r.2.2.1.symm)
      (q.2.2.2.trans r.2.2.2.symm)
  simpa using (Fintype.card_le_one_iff_subsingleton.mpr
  ↪ hsub)

lemma zmod_two_ne_zero {p : ℕ} [Fact p.Prime] (hp2 : p ≠
↪ 2) : (2 : ZMod p) ≠ 0 := by
  intro hzero
  have hpdvd2 : p ∣ 2 := by
    simpa using (ZMod.natCast_eq_zero_iff 2 p).mp hzero
  have hp_le2 : p ≤ 2 := Nat.le_of_dvd (by decide : 0 < 2)
  ↪ hpdvd2
  have hp_ge2 : 2 ≤ p := (Fact.out : Nat.Prime p).two_le
  omega

lemma zmod_two_ne_one {p : ℕ} [Fact p.Prime] : (2 : ZMod
↪ p) ≠ 1 := by
  intro h
  have h' : (2 : ZMod p) - 1 = 0 := by
    simpa using sub_eq_zero.mpr h
  norm_num at h'

lemma card_zmod_excluding_two {p : ℕ} [Fact p.Prime] {x z
↪ : ZMod p} (hxz : x ≠ z) :
   Fintype.card {t : ZMod p // t ≠ x ∧ t ≠ z} = p - 2 := by
  have hEq : Fintype.card {t : ZMod p // t = x ∨ t = z} = 2
  ↪ := by
    simpa [or_comm] using
      (Fintype.card_subtype_eq_or_eq_of_ne (a := x) (b :=
      ↪ z) hxz)
  have hCompl := Fintype.card_subtype_compl (fun t : ZMod
  ↪ p => t = x ∨ t = z)
  simpa [hEq, not_or, ne_comm, and_left_comm, and_assoc]
  ↪ using hCompl

lemma card_excluding_one {α : Type*} [Fintype α]
↪ [DecidableEq α] (x : α) :
   Fintype.card {t : α // t ≠ x} = Fintype.card α - 1 := by
  calc
    Fintype.card {t : α // t ≠ x}
        = Fintype.card α - Fintype.card {t : α // t = x} :=
        ↪ by
            simpa using
```

```
              (Fintype.card_subtype_compl (fun t : α => t =
               ↪  x) :
                Fintype.card {t : α // ¬ t = x} =
                  Fintype.card α - Fintype.card {t : α // t
                    ↪  = x})
    _ = Fintype.card α - 1 := by simp

lemma card_zmod_excluding_one {p : ℕ} [Fact p.Prime] (x :
 ↪  ZMod p) :
    Fintype.card {t : ZMod p // t ≠ x} = p - 1 := by
  simpa [ZMod.card] using card_excluding_one x

lemma card_square_nonzero_excluding_nonsquare {p : ℕ}
 ↪  [Fact p.Prime] (hp2 : p ≠ 2)
    {n : ZMod p} (hn : ¬ IsSquare n) :
    Fintype.card {t : ZMod p // IsSquare t ∧ t ≠ 0 ∧ t ≠ n}
     ↪  = (p - 1) / 2 := by
  classical
  let e : {t : ZMod p // IsSquare t ∧ t ≠ 0 ∧ t ≠ n} ≃
      {t : ZMod p // IsSquare t ∧ t ≠ 0} := {
    toFun := fun t => ⟨t.1, t.2.1, t.2.2.1⟩
    invFun := fun t => ⟨t.1, t.2.1, t.2.2, by
      intro htn
      exact hn (htn ▸ t.2.1)⟩
    left_inv := by intro t; cases t; rfl
    right_inv := by intro t; cases t; rfl
  }
  calc
    Fintype.card {t : ZMod p // IsSquare t ∧ t ≠ 0 ∧ t ≠ n}
        = Fintype.card {t : ZMod p // IsSquare t ∧ t ≠ 0}
         ↪  := Fintype.card_congr e
    _ = (p - 1) / 2 := card_nonzero_square_zmod_odd_prime
     ↪  hp2

lemma card_nonsquare_excluding_nonsquare {p : ℕ} [Fact
 ↪  p.Prime] (hp2 : p ≠ 2)
    {n : ZMod p} (hn : ¬ IsSquare n) :
    Fintype.card {t : ZMod p // ¬ IsSquare t ∧ t ≠ n} = (p
     ↪  - 3) / 2 := by
  classical
  let a : {t : ZMod p // ¬ IsSquare t} := ⟨n, hn⟩
  let e : {t : ZMod p // ¬ IsSquare t ∧ t ≠ n} ≃
      {s : {t : ZMod p // ¬ IsSquare t} // s ≠ a} := {
    toFun := fun t => ⟨⟨t.1, t.2.1⟩, by
      intro hs
      exact t.2.2 (congrArg Subtype.val hs)⟩
    invFun := fun s => ⟨s.1.1, s.1.2, by
      intro hs
```

```
      exact s.2 (Subtype.ext hs)⟩
    left_inv := by intro t; cases t; rfl
    right_inv := by intro s; cases s; rfl
  }
  calc
    Fintype.card {t : ZMod p // ¬ IsSquare t ∧ t ≠ n}
        = Fintype.card {s : {t : ZMod p // ¬ IsSquare t} //
         ↪  s ≠ a} := Fintype.card_congr e
    _ = Fintype.card {t : ZMod p // ¬ IsSquare t} - 1 := by
          simpa using card_excluding_one a
    _ = (p - 1) / 2 - 1 := by rw
     ↪  [card_nonsquare_zmod_odd_prime hp2]
    _ = (p - 3) / 2 := by
          obtain ⟨k, hk⟩ := ((Fact.out : Nat.Prime
           ↪  p).even_sub_one hp2).two_dvd
          omega

lemma quadValue_eq_quadCoeffEval {p : ℕ} (v : QuadCoeff p)
 ↪  (x : ZMod p) :
    quadValue (v 2) (v 1) (v 0) x = quadCoeffEval v x := by
  simp [quadValue, quadCoeffEval, Fin.sum_univ_three]
  ring

lemma goodModFunction_of_rootlessQuad {p : ℕ} [Fact
 ↪  p.Prime] {f : ZMod p → ZMod p}
    (q : RootlessQuad p) (havoid : ∀ x : ZMod p,
     ↪  quadCoeffEval q.1 x + f x ≠ 0) :
    GoodModFunction f := by
  refine ⟨q.1 2, q.1 1, q.1 0, ?_⟩
  intro x hx
  constructor
  · simpa [quadValue_eq_quadCoeffEval] using
   ↪  rootlessQuad_eval_ne_zero q x
  · simpa [quadValue_eq_quadCoeffEval] using havoid x

lemma goodModFunction_zmod_odd_prime_has_zero {p : ℕ}
 ↪  [Fact p.Prime] (hp2 : p ≠ 2)
    {f : ZMod p → ZMod p} (hz : ∃ z, f z = 0) (hnot0 : ∃ x,
     ↪  f x ≠ 0) :
    GoodModFunction f := by
  classical
  rcases hz with ⟨z, hz⟩
  rcases hnot0 with ⟨x₀, hx₀⟩
  have hxz : x₀ ≠ z := by
    intro h
    exact hx₀ (h ▸ hz)
  have h2 : (2 : ZMod p) ≠ 0 := zmod_two_ne_zero hp2
  have h21 : (2 : ZMod p) ≠ 1 := zmod_two_ne_one
```

```
let y₀ : ZMod p := if -f x₀ = 1 then 2 else 1
have hy₀ : y₀ ≠ 0 := by
  by_cases h : -f x₀ = 1
  · simp [y₀, h, h2]
  · simp [y₀, h]
have hy₀bad : y₀ ≠ -f x₀ := by
  by_cases h : -f x₀ = 1
  · simp [y₀, h, h21]
  · simpa [y₀, h, eq_comm] using h
let S := {q : RootlessQuad p // quadCoeffEval q.1 x₀ =
 ↪  y₀}
let BadS : S → Type := fun q => {x : ZMod p //
 ↪  quadCoeffEval q.1.1 x = -f x}
by_cases hall : ∀ q : S, Nonempty (BadS q)
· let R := Σ q : S, BadS q
  let T := {x : ZMod p // x ≠ x₀ ∧ x ≠ z}
  have hRinj :
      Fintype.card S ≤ Fintype.card R := by
    let chooseBad : S → R := fun q => ⟨q,
     ↪  Classical.choice (hall q)⟩
    exact Fintype.card_le_of_injective chooseBad (fun q
     ↪  r h => congrArg Sigma.fst h)
  have hSwap :
      Fintype.card R ≤
        Fintype.card (Σ x : T, {q : S // quadCoeffEval
         ↪  q.1.1 x.1 = -f x.1}) := by
    let g : R → Σ x : T, {q : S // quadCoeffEval q.1.1
     ↪  x.1 = -f x.1} := fun qx =>
      let x : ZMod p := qx.2.1
      have hx0 : x ≠ x₀ := by
        intro hx
        have hbad : y₀ = -f x₀ := by
          calc
            y₀ = quadCoeffEval qx.1.1.1 x₀ := by simpa
             ↪  [S] using qx.1.2.symm
            _ = -f x₀ := by
                  simpa [x, hx] using qx.2.2
        exact hy₀bad hbad
      have hxz' : x ≠ z := by
        intro hx
        have hzero : quadCoeffEval qx.1.1.1 z = 0 := by
          simpa [x, hx, hz] using qx.2.2
        exact rootlessQuad_eval_ne_zero qx.1.1 z hzero
      ⟨⟨x, hx0, hxz'⟩, ⟨qx.1, qx.2.2⟩⟩
    have hg : Function.Injective g := by
      intro qx rx h
      have hx : qx.2.1 = rx.2.1 := congrArg (fun s =>
       ↪  ((Sigma.fst s : T) : ZMod p)) h
```

```
      have hq : qx.1 = rx.1 := congrArg (fun s =>
       ↪  (Sigma.snd s).1) h
      cases qx
      cases rx
      cases hq
      apply Sigma.ext
      · rfl
      · exact (Subtype.ext hx).heq
    exact Fintype.card_le_of_injective g hg
  have hFiber :
      ∀ x : T, Fintype.card {q : S // quadCoeffEval q.1.1
       ↪  x.1 = -f x.1} ≤ (p + 1) / 2 := by
    intro x
    by_cases hx0 : f x.1 = 0
    · have hEmpty : IsEmpty {q : S // quadCoeffEval q.1.1
     ↪  x.1 = -f x.1} := by
        refine ⟨?_⟩
        intro q
        have hzero : quadCoeffEval q.1.1.1 x.1 = 0 := by
         ↪  simpa [hx0] using q.2
        exact rootlessQuad_eval_ne_zero q.1.1 x.1 hzero
      have hcard : Fintype.card {q : S // quadCoeffEval
       ↪  q.1.1 x.1 = -f x.1} = 0 :=
        Fintype.card_eq_zero_iff.mpr hEmpty
      omega
    · have hy : (-f x.1 : ZMod p) ≠ 0 := neg_ne_zero.mpr
     ↪  hx0
      let g :
          {q : S // quadCoeffEval q.1.1 x.1 = -f x.1} →
            {q : RootlessQuad p //
              quadCoeffEval q.1 x₀ = y₀ ∧ quadCoeffEval
               ↪  q.1 x.1 = -f x.1} := fun q =>
                ⟨q.1.1, q.1.2, q.2⟩
      have hg : Function.Injective g := by
        intro q r h
        apply Subtype.ext
        apply Subtype.ext
        exact congrArg
          (fun s :
            {q : RootlessQuad p //
              quadCoeffEval q.1 x₀ = y₀ ∧ quadCoeffEval
               ↪  q.1 x.1 = -f x.1} =>
                (s : RootlessQuad p)) h
      exact (Fintype.card_le_of_injective g hg).trans
        (card_rootlessQuad_eval_pair_le hp2 x.2.1.symm
         ↪  hy₀ hy)
  have hSigma :
```

```
    Fintype.card (Σ x : T, {q : S // quadCoeffEval
     ↪  q.1.1 x.1 = -f x.1}) ≤
      Fintype.card T * ((p + 1) / 2) := by
  rw [Fintype.card_sigma]
  calc
    Σ x : T, Fintype.card {q : S // quadCoeffEval q.1.1
     ↪  x.1 = -f x.1}
        ≤ Σ _x : T, (p + 1) / 2 := Finset.sum_le_sum
         ↪  (fun x hx => hFiber x)
    _ = Fintype.card T * ((p + 1) / 2) := by
          simpa using
            (show Fintype.card ↥(Finset.univ : Finset
             ↪  T) * ((p + 1) / 2) =
                Fintype.card T * ((p + 1) / 2) by
              simp)
have hT : Fintype.card T = p - 2 := by
  simpa [T] using card_zmod_excluding_two (p := p) hxz
have hS : Fintype.card S = p * ((p - 1) / 2) := by
  simpa [S] using card_rootlessQuad_eval_eq hp2 x₀ y₀
   ↪  hy₀
have : p * ((p - 1) / 2) ≤ (p - 2) * ((p + 1) / 2) := by
  calc
    p * ((p - 1) / 2) = Fintype.card S := hS.symm
    _ ≤ Fintype.card R := hRinj
    _ ≤ Fintype.card (Σ x : T, {q : S // quadCoeffEval
     ↪  q.1.1 x.1 = -f x.1}) := hSwap
    _ ≤ Fintype.card T * ((p + 1) / 2) := hSigma
    _ = (p - 2) * ((p + 1) / 2) := by rw [hT]
have hfalse : ¬p * ((p - 1) / 2) ≤ (p - 2) * ((p + 1) /
 ↪  2) := by
  intro hineq
  obtain ⟨k, hk⟩ := ((Fact.out : Nat.Prime
   ↪  p).even_sub_one hp2).two_dvd
  have hk1 : 1 ≤ k := by
    have hp_ge2 : 2 ≤ p := (Fact.out : Nat.Prime
     ↪  p).two_le
    omega
  have hpodd : p = 2 * k + 1 := by
    omega
  have hineq' : (2 * k + 1) * k ≤ (2 * k - 1) * (k + 1)
   ↪  := by
    calc
      (2 * k + 1) * k = p * ((p - 1) / 2) := by
            rw [hpodd]
            norm_num
      _ ≤ (p - 2) * ((p + 1) / 2) := hineq
      _ = (2 * k - 1) * (k + 1) := by
            rw [hpodd]
```

```
                have hsub : 2 * k + 1 - 2 = 2 * k - 1 := by
                  omega
                have hdiv : (2 * k + 1 + 1) / 2 = k + 1 := by
                  omega
                rw [hsub, hdiv]
      have hineqZ :
          (((2 * k + 1) * k : ℕ) : ℤ) ≤ (((2 * k - 1) * (k
          ↪  + 1) : ℕ) : ℤ) := by
        exact_mod_cast hineq'
      have hkminus : (((2 * k - 1 : ℕ) : ℤ)) = 2 * k - 1 :=
       ↪  by
        omega
      norm_num at hineqZ
      rw [hkminus] at hineqZ
      nlinarith
    exfalso
    exact hfalse this
  · push Not at hall
    rcases hall with ⟨q, hq⟩
    refine goodModFunction_of_rootlessQuad q.1 ?_
    intro x hx
    exact hq.false ⟨x, by simpa [eq_neg_iff_add_eq_zero]
     ↪  using hx⟩

lemma goodModFunction_zmod_odd_prime_nowhere_zero_surject⌋
 ↪  ive {p : ℕ} [Fact
 ↪  p.Prime]
    (hp2 : p ≠ 2) (_hp3 : p ≠ 3) {f : ZMod p → ZMod p}
    (hnozero : ∀ x : ZMod p, f x ≠ 0)
    (hsurj : ∀ y : ZMod p, y ≠ 0 → ∃ x : ZMod p, f x = y) :
    GoodModFunction f := by
  classical
  have hchar : ringChar (ZMod p) ≠ 2 := by
    simpa [ZMod.ringChar_zmod_n p] using hp2
  obtain ⟨n, hnsq⟩ := FiniteField.exists_nonsquare (F :=
   ↪  ZMod p) hchar
  have hn0 : n ≠ 0 := by
    intro hn
    exact hnsq (hn ▸ IsSquare.zero)
  let pre : {y : ZMod p // y ≠ 0} → ZMod p := fun y =>
   ↪  Classical.choose (hsurj y.1 y.2)
  have hpre : ∀ y : {y : ZMod p // y ≠ 0}, f (pre y) = y.1
   ↪  := by
    intro y
    exact Classical.choose_spec (hsurj y.1 y.2)
  have hpre_inj : Function.Injective pre := by
    intro y z h
    apply Subtype.ext
```

```
  calc
    y.1 = f (pre y) := (hpre y).symm
    _ = f (pre z) := by rw [h]
    _ = z.1 := hpre z
have hpre_not_surj : ¬ Function.Surjective pre := by
  intro hsurj_pre
  have hlt : Fintype.card {y : ZMod p // y ≠ 0} <
   ↪  Fintype.card (ZMod p) := by
    rw [card_nonzero_zmod (p := p), ZMod.card]
    exact Nat.sub_lt ((Fact.out : Nat.Prime p).pos) (by
     ↪  decide : 0 < 1)
  have hle : Fintype.card (ZMod p) ≤ Fintype.card {y :
   ↪  ZMod p // y ≠ 0} :=
    Fintype.card_le_of_surjective pre hsurj_pre
  exact Nat.not_le_of_lt hlt hle
have hpre_not_surj' : ∃ xExtra : ZMod p, ∀ y : {y : ZMod
 ↪  p // y ≠ 0}, pre y ≠ xExtra := by
  simpa [Function.Surjective] using hpre_not_surj
rcases hpre_not_surj' with ⟨xExtra, hxExtra⟩
have hxExtra0 : f xExtra ≠ 0 := hnozero xExtra
let y0 : ZMod p := -f xExtra
have hy0 : y0 ≠ 0 := neg_ne_zero.mpr hxExtra0
let u : ZMod p := n / f xExtra
have hu0 : u ≠ 0 := by
  exact div_ne_zero hn0 hxExtra0
let x0 : ZMod p := pre ⟨u, hu0⟩
have hx0val : f x0 = u := by
  simpa [x0, pre] using hpre ⟨u, hu0⟩
have hx0Extra : x0 ≠ xExtra := by
  simpa [x0] using hxExtra ⟨u, hu0⟩
have hy0u : y0 * (-u) = n := by
  dsimp [y0, u]
  field_simp [hxExtra0]
have hy0bad : y0 ≠ -f x0 := by
  intro h
  have hsquare : IsSquare n := by
    refine ⟨u, ?_⟩
    calc
      n = y0 * (-u) := hy0u.symm
      _ = (-f x0) * (-u) := by rw [h]
      _ = u * u := by simp [hx0val]
  exact hnsq hsquare
let X : Type := {x : ZMod p // x ≠ xExtra}
let Y : Type := {y : ZMod p // y ≠ 0}
let preX : Y → X := fun y => ⟨pre y, hxExtra y⟩
have hpreX_inj : Function.Injective preX := by
  intro y z h
  apply Subtype.ext
```

```
    exact congrArg Subtype.val (hpre_inj (by simpa [preX]
     ↪  using congrArg Subtype.val h))
  have hcardXY : Fintype.card Y = Fintype.card X := by
    simp [X, Y, card_nonzero_zmod, card_zmod_excluding_one]
  have hpreX_bij : Function.Bijective preX := by
    exact (Fintype.bijective_iff_injective_and_card
     ↪  preX).2 ⟨hpreX_inj, hcardXY⟩
  let fX : X → Y := fun x => ⟨f x.1, hnozero x.1⟩
  have hfX_preX : Function.LeftInverse fX preX := by
    intro y
    apply Subtype.ext
    simpa [preX, fX] using hpre y
  have hpreX_fX : Function.LeftInverse preX fX := by
    intro x
    rcases hpreX_bij.2 x with ⟨y, rfl⟩
    exact congrArg preX (hfX_preX y)
  let S := {q : RootlessQuad p // quadCoeffEval q.1 x0 =
   ↪  y0}
  let BadS : S → Type := fun q =>
    {x : ZMod p // x ≠ x0 ∧ quadCoeffEval q.1.1 x = -f x}
  by_cases hall : ∀ q : S, Nonempty (BadS q)
  · let Rextra := {q : S // quadCoeffEval q.1.1 xExtra =
   ↪  -f xExtra}
    let Yu : Type := {y : ZMod p // y ≠ 0 ∧ y ≠ u}
    let Fiber : Yu → Type := fun y =>
      {q : S // quadCoeffEval q.1.1 (pre ⟨y.1, y.2.1⟩) =
       ↪  -y.1}
    have hSinj :
        Fintype.card S ≤
          Fintype.card (Rextra ⊕ Σ y : Yu, Fiber y) := by
      let g : S → (Rextra ⊕ Σ y : Yu, Fiber y) := fun q =>
        let x := Classical.choice (hall q)
        if hxe : x.1 = xExtra then
          Sum.inl ⟨q, by simpa [hxe] using x.2.2⟩
        else
          let y : Y := fX ⟨x.1, hxe⟩
          have hyx : pre y = x.1 := by
            simpa [y, preX, fX] using congrArg Subtype.val
             ↪  (hpreX_fX ⟨x.1, hxe⟩)
          have hyu : y.1 ≠ u := by
            intro hyu
            have hyEq : y = ⟨u, hu0⟩ := by
              apply Subtype.ext
              exact hyu
            have hxeq : x.1 = x0 := by
              have : pre ⟨u, hu0⟩ = x.1 := by simpa [hyEq]
               ↪  using hyx
              simpa [x0] using this.symm
```

```
        exact x.2.1 hxeq
      Sum.inr ⟨⟨y.1, y.2, hyu⟩, ⟨q, by
        simpa [y, fX, hyx] using x.2.2⟩⟩
  let recover : (Rextra ⊕ Σ y : Yu, Fiber y) → S :=
    Sum.elim Subtype.val (fun s => s.2.1)
  have hrecover : Function.LeftInverse recover g := by
    intro q
    dsimp [recover, g]
    split_ifs <;> rfl
  exact Fintype.card_le_of_injective g
   ↪ hrecover.injective
have hRextra :
    Fintype.card Rextra ≤ (p - 1) / 2 := by
  let g :
      Rextra →
        {q : RootlessQuad p //
         quadCoeffEval q.1 x0 = y0 ∧ quadCoeffEval q.1
          ↪ xExtra = -f xExtra} := fun q =>
           ⟨q.1.1, q.1.2, q.2⟩
  have hg : Function.Injective g := by
    intro q r h
    apply Subtype.ext
    apply Subtype.ext
    exact congrArg
      (fun s :
        {q : RootlessQuad p //
         quadCoeffEval q.1 x0 = y0 ∧ quadCoeffEval q.1
          ↪ xExtra = -f xExtra} =>
           (s : RootlessQuad p)) h
  have hsqExtra : IsSquare (y0 * (-f xExtra)) := by
    refine ⟨-f xExtra, ?_⟩
    simp [y0, pow_two]
  exact (Fintype.card_le_of_injective g hg).trans
    (card_rootlessQuad_eval_pair_le_square hp2
     ↪ hx0Extra hy0
      (neg_ne_zero.mpr hxExtra0) hsqExtra)
have hFiberSq :
    ∀ y : {y : Yu // IsSquare (y0 * (-y.1))},
     ↪ Fintype.card (Fiber y.1) ≤ (p - 1) / 2 := by
  intro y
  have hy : (-y.1.1 : ZMod p) ≠ 0 := neg_ne_zero.mpr
   ↪ y.1.2.1
  have hyx0 : x0 ≠ pre ⟨y.1.1, y.1.2.1⟩ := by
    intro h
    have hsub : (⟨y.1.1, y.1.2.1⟩ : Y) = ⟨u, hu0⟩ := by
      exact hpre_inj (by simpa [x0] using h.symm)
    exact y.1.2.2 (congrArg Subtype.val hsub)
  let g :
```

```
        Fiber y.1 →
          {q : RootlessQuad p //
           quadCoeffEval q.1 x0 = y0 ∧ quadCoeffEval q.1
            ↪  (pre ⟨y.1.1, y.1.2.1⟩) = -y.1.1} := fun
            ↪  q =>
             ⟨q.1.1, q.1.2, q.2⟩
    have hg : Function.Injective g := by
      intro q r h
      apply Subtype.ext
      apply Subtype.ext
      exact congrArg
        (fun s :
          {q : RootlessQuad p //
           quadCoeffEval q.1 x0 = y0 ∧ quadCoeffEval q.1
            ↪  (pre ⟨y.1.1, y.1.2.1⟩) = -y.1.1} =>
             (s : RootlessQuad p)) h
    exact (Fintype.card_le_of_injective g hg).trans
      (card_rootlessQuad_eval_pair_le_square hp2 hyx0
       ↪  hy0 hy y.2)
  have hFiberNonsq :
      ∀ y : {y : Yu // ¬ IsSquare (y0 * (-y.1))},
       ↪  Fintype.card (Fiber y.1) ≤ (p + 1) / 2 := by
    intro y
    have hy : (-y.1.1 : ZMod p) ≠ 0 := neg_ne_zero.mpr
     ↪  y.1.2.1
    have hyx0 : x0 ≠ pre ⟨y.1.1, y.1.2.1⟩ := by
      intro h
      have hsub : (⟨y.1.1, y.1.2.1⟩ : Y) = ⟨u, hu0⟩ := by
        exact hpre_inj (by simpa [x0] using h.symm)
      exact y.1.2.2 (congrArg Subtype.val hsub)
    let g :
        Fiber y.1 →
          {q : RootlessQuad p //
           quadCoeffEval q.1 x0 = y0 ∧ quadCoeffEval q.1
            ↪  (pre ⟨y.1.1, y.1.2.1⟩) = -y.1.1} := fun
            ↪  q =>
             ⟨q.1.1, q.1.2, q.2⟩
    have hg : Function.Injective g := by
      intro q r h
      apply Subtype.ext
      apply Subtype.ext
      exact congrArg
        (fun s :
          {q : RootlessQuad p //
           quadCoeffEval q.1 x0 = y0 ∧ quadCoeffEval q.1
            ↪  (pre ⟨y.1.1, y.1.2.1⟩) = -y.1.1} =>
             (s : RootlessQuad p)) h
    exact (Fintype.card_le_of_injective g hg).trans
```

```
    (card_rootlessQuad_eval_pair_le hp2 hyx0 hy0 hy)
have hYuSq :
    Fintype.card {y : Yu // IsSquare (y0 * (-y.1))} =
     ↪ (p - 1) / 2 := by
  let emul : Yu ≃ {t : ZMod p // t ≠ 0 ∧ t ≠ n} := {
    toFun := fun y => ⟨y0 * (-y.1), mul_ne_zero hy0
     ↪ (neg_ne_zero.mpr y.2.1), by
      intro h
      have hm : y0 * (-y.1) = y0 * (-u) := by rw [h,
       ↪ hy0u]
      have hneg : -y.1 = -u := mul_left_cancel₀ hy0 hm
      exact y.2.2 (neg_injective hneg)⟩
    invFun := fun t => ⟨-t.1 / y0, by
      constructor
      · simpa using div_ne_zero (neg_ne_zero.mpr t.2.1)
       ↪ hy0
      · intro h
        have ht : t.1 = n := by
          calc
            t.1 = y0 * (-(-t.1 / y0)) := by
              field_simp [hy0, t.2.1]
            _ = y0 * (-u) := by rw [h]
            _ = n := hy0u
        exact t.2.2 ht⟩
    left_inv := by
      intro y
      apply Subtype.ext
      dsimp
      field_simp [hy0, y.2.1]
    right_inv := by
      intro t
      apply Subtype.ext
      dsimp
      field_simp [hy0, t.2.1]
  }
  let e1 : {y : Yu // IsSquare (y0 * (-y.1))} ≃
      {t : {t : ZMod p // t ≠ 0 ∧ t ≠ n} // IsSquare
       ↪ t.1} := {
    toFun := fun y => ⟨emul y.1, by simpa [emul] using
     ↪ y.2⟩
    invFun := fun t => ⟨emul.symm t.1, by
      change IsSquare ((emul (emul.symm t.1)).1)
      simpa using t.2⟩
    left_inv := by intro y; apply Subtype.ext; exact
     ↪ emul.left_inv y.1
    right_inv := by intro t; apply Subtype.ext; exact
     ↪ emul.right_inv t.1
  }
```

```
    let e2 : {t : {t : ZMod p // t ≠ 0 ∧ t ≠ n} //
     ↪  IsSquare t.1} ≃
        {t : ZMod p // IsSquare t ∧ t ≠ 0 ∧ t ≠ n} := {
      toFun := fun t => ⟨t.1.1, t.2, t.1.2.1, t.1.2.2⟩
      invFun := fun t => ⟨⟨t.1, t.2.2.1, t.2.2.2⟩, t.2.1⟩
      left_inv := by intro t; cases t; rfl
      right_inv := by intro t; cases t; rfl
    }
    calc
      Fintype.card {y : Yu // IsSquare (y0 * (-y.1))}
          = Fintype.card {t : ZMod p // IsSquare t ∧ t ≠
           ↪  0 ∧ t ≠ n} := by
              exact Fintype.card_congr (e1.trans e2)
      _ = (p - 1) / 2 :=
       ↪  card_square_nonzero_excluding_nonsquare hp2
       ↪  hnsq
  have hYuNonsq :
      Fintype.card {y : Yu // ¬ IsSquare (y0 * (-y.1))} =
       ↪  (p - 3) / 2 := by
    let emul : Yu ≃ {t : ZMod p // t ≠ 0 ∧ t ≠ n} := {
      toFun := fun y => ⟨y0 * (-y.1), mul_ne_zero hy0
       ↪  (neg_ne_zero.mpr y.2.1), by
        intro h
        have hm : y0 * (-y.1) = y0 * (-u) := by rw [h,
         ↪  hy0u]
        have hneg : -y.1 = -u := mul_left_cancel₀ hy0 hm
        exact y.2.2 (neg_injective hneg)⟩
      invFun := fun t => ⟨-t.1 / y0, by
        constructor
        · simpa using div_ne_zero (neg_ne_zero.mpr t.2.1)
         ↪  hy0
        · intro h
          have ht : t.1 = n := by
            calc
              t.1 = y0 * (-(-t.1 / y0)) := by
                field_simp [hy0, t.2.1]
              _ = y0 * (-u) := by rw [h]
              _ = n := hy0u
          exact t.2.2 ht⟩
      left_inv := by
        intro y
        apply Subtype.ext
        dsimp
        field_simp [hy0, y.2.1]
      right_inv := by
        intro t
        apply Subtype.ext
        dsimp
```

```
      field_simp [hy0, t.2.1]
  }
  let e1 : {y : Yu // ¬ IsSquare (y0 * (-y.1))} ≃
      {t : {t : ZMod p // t ≠ 0 ∧ t ≠ n} // ¬ IsSquare
       ↪  t.1} := {
    toFun := fun y => ⟨emul y.1, by simpa [emul] using
     ↪  y.2⟩
    invFun := fun t => ⟨emul.symm t.1, by
      change ¬ IsSquare ((emul (emul.symm t.1)).1)
      simpa using t.2⟩
    left_inv := by intro y; apply Subtype.ext; exact
     ↪  emul.left_inv y.1
    right_inv := by intro t; apply Subtype.ext; exact
     ↪  emul.right_inv t.1
  }
  let e2 : {t : {t : ZMod p // t ≠ 0 ∧ t ≠ n} // ¬
   ↪  IsSquare t.1} ≃
      {t : ZMod p // ¬ IsSquare t ∧ t ≠ n} := {
    toFun := fun t => ⟨t.1.1, t.2, t.1.2.2⟩
    invFun := fun t => ⟨⟨t.1, by
      constructor
      · intro ht0
        exact t.2.1 (ht0 ▸ IsSquare.zero)
      · exact t.2.2⟩, t.2.1⟩
    left_inv := by intro t; cases t; rfl
    right_inv := by intro t; cases t; rfl
  }
  calc
    Fintype.card {y : Yu // ¬ IsSquare (y0 * (-y.1))}
        = Fintype.card {t : ZMod p // ¬ IsSquare t ∧ t
         ↪  ≠ n} := by
            exact Fintype.card_congr (e1.trans e2)
    _ = (p - 3) / 2 :=
     ↪  card_nonsquare_excluding_nonsquare hp2 hnsq
have hSigma :
    Fintype.card (Σ y : Yu, Fiber y) ≤
      ((p - 1) / 2) * ((p - 1) / 2) + ((p - 3) / 2) *
       ↪  ((p + 1) / 2) := by
  let SqY : Type := {y : Yu // IsSquare (y0 * (-y.1))}
  let NsqY : Type := {y : Yu // ¬ IsSquare (y0 *
   ↪  (-y.1))}
  let Total : Type := Σ y : Yu, Fiber y
  have hsplit :
      Fintype.card Total =
        Fintype.card {s : Total // IsSquare (y0 *
         ↪  (-s.1.1))} +
```

```
        Fintype.card {s : Total // ¬ IsSquare (y0 *
         ↪  (-s.1.1))} := by
  rw [Fintype.card_subtype_compl (p := fun s : Total
   ↪  => IsSquare (y0 * (-s.1.1)))]
  exact (Nat.add_sub_of_le
    (Fintype.card_subtype_le (fun s : Total =>
     ↪  IsSquare (y0 * (-s.1.1))))).symm
have hSqCard :
    Fintype.card {s : Total // IsSquare (y0 *
     ↪  (-s.1.1))} =
      Fintype.card (Σ y : SqY, Fiber y.1) := by
  change Fintype.card {s : (Σ y : Yu, Fiber y) //
   ↪  IsSquare (y0 * (-s.1.1))} =
    Fintype.card (Σ y : SqY, Fiber y.1)
  exact Fintype.card_congr
    (Equiv.subtypeSigmaEquiv Fiber
      (fun y : Yu => IsSquare (y0 * (-y.1))))
have hNsqCard :
    Fintype.card {s : Total // ¬ IsSquare (y0 *
     ↪  (-s.1.1))} =
      Fintype.card (Σ y : NsqY, Fiber y.1) := by
  change Fintype.card {s : (Σ y : Yu, Fiber y) // ¬
   ↪  IsSquare (y0 * (-s.1.1))} =
    Fintype.card (Σ y : NsqY, Fiber y.1)
  exact Fintype.card_congr
    (Equiv.subtypeSigmaEquiv Fiber
      (fun y : Yu => ¬ IsSquare (y0 * (-y.1))))
have hSqSigma :
    Fintype.card (Σ y : SqY, Fiber y.1) ≤ ((p - 1) /
     ↪  2) * ((p - 1) / 2) := by
  rw [Fintype.card_sigma]
  calc
    Σ y : SqY, Fintype.card (Fiber y.1) ≤ Σ _y : SqY,
     ↪  (p - 1) / 2 := by
      refine Finset.sum_le_sum ?_
      intro y hy
      exact hFiberSq y
    _ = Fintype.card SqY * ((p - 1) / 2) := by simp
    _ = ((p - 1) / 2) * ((p - 1) / 2) := by
          simpa [SqY] using congrArg (fun n => n *
           ↪  ((p - 1) / 2)) hYuSq
have hNsqSigma :
    Fintype.card (Σ y : NsqY, Fiber y.1) ≤ ((p - 3) /
     ↪  2) * ((p + 1) / 2) := by
  rw [Fintype.card_sigma]
  calc
```

```
      Σ y : NsqY, Fintype.card (Fiber y.1) ≤ Σ _y :
       ↪  NsqY, (p + 1) / 2 := by
        refine Finset.sum_le_sum ?_
        intro y hy
        exact hFiberNonsq y
      _ = Fintype.card NsqY * ((p + 1) / 2) := by simp
      _ = ((p - 3) / 2) * ((p + 1) / 2) := by
            simpa [NsqY] using congrArg (fun n => n *
             ↪  ((p + 1) / 2)) hYuNonsq
  have hsplit' :
      Fintype.card (Σ y : Yu, Fiber y) =
        Fintype.card (Σ y : SqY, Fiber y.1) +
         ↪  Fintype.card (Σ y : NsqY, Fiber y.1) := by
    rw [hSqCard, hNsqCard] at hsplit
    simpa [Total] using hsplit
  calc
    Fintype.card (Σ y : Yu, Fiber y)
        = Fintype.card (Σ y : SqY, Fiber y.1) +
         ↪  Fintype.card (Σ y : NsqY, Fiber y.1) :=
         ↪  hsplit'
    _ ≤ ((p - 1) / 2) * ((p - 1) / 2) + ((p - 3) / 2) *
     ↪  ((p + 1) / 2) := by
          exact add_le_add hSqSigma hNsqSigma
have hS : Fintype.card S = p * ((p - 1) / 2) := by
  simpa [S] using card_rootlessQuad_eval_eq hp2 x0 y0
   ↪  hy0
have hbound :
    p * ((p - 1) / 2) ≤
      (p - 1) / 2 + ((p - 1) / 2) * ((p - 1) / 2) + ((p
       ↪  - 3) / 2) * ((p + 1) / 2) := by
  calc
    p * ((p - 1) / 2) = Fintype.card S := hS.symm
    _ ≤ Fintype.card (Rextra ⊕ Σ y : Yu, Fiber y) :=
     ↪  hSinj
    _ = Fintype.card Rextra + Fintype.card (Σ y : Yu,
     ↪  Fiber y) := by
          rw [Fintype.card_sum]
    _ ≤ (p - 1) / 2 + (((p - 1) / 2) * ((p - 1) / 2) +
     ↪  ((p - 3) / 2) * ((p + 1) / 2)) := by
          exact add_le_add hRextra hSigma
    _ = (p - 1) / 2 + ((p - 1) / 2) * ((p - 1) / 2) +
     ↪  ((p - 3) / 2) * ((p + 1) / 2) := by
          simp [add_assoc]
have hfalse :
    ¬ p * ((p - 1) / 2) ≤
      (p - 1) / 2 + ((p - 1) / 2) * ((p - 1) / 2) + ((p
       ↪  - 3) / 2) * ((p + 1) / 2) := by
```

```
      intro hineq
      obtain ⟨k, hk⟩ := ((Fact.out : Nat.Prime
       ↪  p).even_sub_one hp2).two_dvd
      have hkge1 : 1 ≤ k := by
        have hp_ge2 : 2 ≤ p := (Fact.out : Nat.Prime
         ↪  p).two_le
        omega
      have hpodd : p = 2 * k + 1 := by
        omega
      have hhalf1 : (2 * k + 1 - 1) / 2 = k := by omega
      have hhalf2 : (2 * k + 1 - 3) / 2 = k - 1 := by omega
      have hhalf3 : (2 * k + 1 + 1) / 2 = k + 1 := by omega
      rw [hpodd] at hineq
      rw [hhalf1, hhalf2, hhalf3] at hineq
      have hkge1Z : (1 : ℤ) ≤ k := by exact_mod_cast hkge1
      have hineqZ :
          (((2 * k + 1) * k : ℕ) : ℤ) ≤ (((k + k * k + (k -
           ↪  1) * (k + 1)) : ℕ) : ℤ) := by
        exact_mod_cast hineq
      have hkminusZ : (((k - 1 : ℕ) : ℤ)) = k - 1 := by
       ↪  omega
      norm_num at hineqZ
      rw [hkminusZ] at hineqZ
      ring_nf at hineqZ
      linarith
    exfalso
    exact hfalse hbound
  · obtain ⟨q, hq⟩ := not_forall.mp hall
    refine goodModFunction_of_rootlessQuad q.1 ?_
    intro x hx
    by_cases hx0' : x = x0
    · have hbad0 : quadCoeffEval q.1 x0 = -f x0 := by
        simpa [hx0'] using (eq_neg_iff_add_eq_zero.mpr hx)
      exact hy0bad (q.2.symm.trans hbad0)
    · have hbadx : quadCoeffEval q.1 x = -f x :=
          eq_neg_iff_add_eq_zero.mpr hx
      exact hq ⟨⟨x, hx0', hbadx⟩⟩

lemma goodModProduct_zmod_odd_prime {p : ℕ} [Fact p.Prime]
 ↪  (hp2 : p ≠ 2) :
    ∀ f : ZMod p → ZMod p, GoodModProduct f := by
  intro f
  by_cases hz : ∃ z, f z = 0
  · by_cases hnot0 : ∃ x, f x ≠ 0
    · exact (goodModFunction_zmod_odd_prime_has_zero hp2
     ↪  hz hnot0).to_product
    · push Not at hnot0
      have hf0 : f = fun _ : ZMod p => 0 := by
```

```
        funext x
        exact hnot0 x
      simpa [hf0] using
        (goodModFunction_zmod_odd_prime_zero
         ↪ hp2).to_product
  · have hnozero : ∀ x : ZMod p, f x ≠ 0 := by
      intro x hx
      exact hz ⟨x, hx⟩
    by_cases hsurj : ∀ y : ZMod p, y ≠ 0 → ∃ x : ZMod p, f
     ↪ x = y
    · by_cases hp3 : p = 3
      · subst hp3
        exact goodModProduct_zmod_three f
      · exact (goodModFunction_zmod_odd_prime_nowhere_zer
       ↪ o_surjective hp2 hp3 hnozero
       ↪ hsurj).to_product
    · exact
     ↪ (goodModFunction_zmod_of_not_surjective_nonzero
     ↪ hsurj).to_product

lemma allPolynomialsNGood_odd_prime {p : ℕ} [Fact p.Prime]
 ↪ (hp2 : p ≠ 2) :
    AllPolynomialsNGood p := by
  intro P
  exact polynomialNGood_of_goodModProduct P
    (goodModProduct_zmod_odd_prime hp2 (polynomialModFn p
     ↪ P))

theorem allPolynomialsNGood_iff_of_pos {n : ℕ} (hn : 0 <
 ↪ n) :
    AllPolynomialsNGood n ↔ 3 ≤ n := by
  constructor
  · intro hgood
    have hn1 : n ≠ 1 := by
      intro h1
      subst h1
      exact not_allPolynomialsNGood_one hgood
    have hn2 : n ≠ 2 := by
      intro h2
      subst h2
      exact not_allPolynomialsNGood_two hgood
    omega
  · intro hn3
    by_cases h3 : 3 ∣ n
    · exact AllPolynomialsNGood.of_dvd h3
     ↪ allPolynomialsNGood_three
    · by_cases h4 : 4 ∣ n
```

```
      · exact AllPolynomialsNGood.of_dvd h4
       ↪  allPolynomialsNGood_four
      · by_cases heven : Even n
        · rcases heven with ⟨m, rfl⟩
          have hmgt1 : m ≠ 1 := by
            intro hm1
            subst hm1
            omega
          obtain ⟨p, hpprime, hpdvdm⟩ :=
           ↪  Nat.exists_prime_and_dvd hmgt1
          have hp2 : p ≠ 2 := by
            intro hp2
            subst hp2
            apply h4
            rcases hpdvdm with ⟨k, hk⟩
            refine ⟨k, ?_⟩
            omega
          have hp3 : p ≠ 3 := by
            intro hp3
            subst hp3
            apply h3
            simpa [two_mul] using dvd_mul_of_dvd_right
             ↪  hpdvdm 2
          letI : Fact p.Prime := ⟨hpprime⟩
          exact AllPolynomialsNGood.of_dvd (by simpa
           ↪  [two_mul] using dvd_mul_of_dvd_right hpdvdm
           ↪  2)
            (allPolynomialsNGood_odd_prime hp2)
        · have hn1 : n ≠ 1 := by omega
          obtain ⟨p, hpprime, hpdvdn⟩ :=
           ↪  Nat.exists_prime_and_dvd hn1
          have hp2 : p ≠ 2 := by
            intro hp2
            subst hp2
            exact heven (even_iff_two_dvd.mpr hpdvdn)
          have hp3 : p ≠ 3 := by
            intro hp3
            subst hp3
            exact h3 hpdvdn
          letI : Fact p.Prime := ⟨hpprime⟩
          exact AllPolynomialsNGood.of_dvd hpdvdn
           ↪  (allPolynomialsNGood_odd_prime hp2)

lemma card_quadCoeff_eval_eq {p : ℕ} [Fact p.Prime] (x y :
 ↪  ZMod p) :
    Fintype.card {v : QuadCoeff p // quadCoeffEval v x =
     ↪  y} = p ^ 2 := by
  classical
```

```
  let e : {v : QuadCoeff p // quadCoeffEval v x = y} ≃ ZMod
   ↪  p × ZMod p := {
    toFun := fun v => (v.1 1, v.1 2)
    invFun := fun bc =>
      ⟨![y - bc.1 * x - bc.2 * x ^ 2, bc.1, bc.2], by
        simp [quadCoeffEval, Fin.sum_univ_three]
        ring⟩
    left_inv := by
      intro v
      ext i
      fin_cases i
      · change y - v.1 1 * x - v.1 2 * x ^ 2 = v.1 0
       have hv : y = v.1 0 + v.1 1 * x + v.1 2 * x ^ 2 := by
          simpa [quadCoeffEval, Fin.sum_univ_three] using
           ↪  v.2.symm
        calc
          y - v.1 1 * x - v.1 2 * x ^ 2
              = (v.1 0 + v.1 1 * x + v.1 2 * x ^ 2) - v.1 1
               ↪  * x - v.1 2 * x ^ 2 := by
                  simp [hv]
          _ = v.1 0 := by ring
      · change v.1 1 = v.1 1
        simp
      · change v.1 2 = v.1 2
        simp
    right_inv := by
      intro bc
      ext <;> simp
  }
  simpa [QuadCoeff, pow_two] using Fintype.card_congr e

section Counting

variable {α : Type*} [Fintype α] [DecidableEq α]

lemma card_orderedPairs_ne :
    Fintype.card {xy : α × α // xy.1 ≠ xy.2} =
     ↪  Fintype.card α * (Fintype.card α - 1) := by
  classical
  let e : {xy : α × α // xy.1 ≠ xy.2} ≃ Σ x : α, {y : α //
   ↪  y ≠ x} := {
    toFun := fun xy => ⟨xy.1.1, ⟨xy.1.2, by simpa [ne_comm]
     ↪  using xy.2⟩⟩
    invFun := fun xy => ⟨(xy.1, xy.2.1), by simpa [ne_comm]
     ↪  using xy.2.2⟩
    left_inv := by intro xy; cases xy; rfl
    right_inv := by intro xy; cases xy; rfl
  }
```

```
  rw [Fintype.card_congr e, Fintype.card_sigma]
  have hcard : ∀ x : α, Fintype.card {y : α // y ≠ x} =
   ↪  Fintype.card α - 1 := by
    intro x
    calc
      Fintype.card {y : α // y ≠ x}
          = Fintype.card α - Fintype.card {y : α // y = x}
           ↪  := by
              simpa using
                (Fintype.card_subtype_compl (fun y : α => y
                 ↪  = x) :
                  Fintype.card {y : α // ¬y = x} =
                   Fintype.card α - Fintype.card {y : α //
                    ↪  y = x})
      _ = Fintype.card α - 1 := by simp
  simp [hcard]

lemma card_orderedTriples_ne :
    Fintype.card {xyz : α × α × α // xyz.1 ≠ xyz.2.1 ∧
     ↪  xyz.1 ≠ xyz.2.2 ∧ xyz.2.1 ≠ xyz.2.2} =
      Fintype.card α * (Fintype.card α - 1) *
       ↪  (Fintype.card α - 2) := by
  classical
  let e :
      {xyz : α × α × α // xyz.1 ≠ xyz.2.1 ∧ xyz.1 ≠ xyz.2.2
       ↪  ∧ xyz.2.1 ≠ xyz.2.2} ≃
        Σ x : α, Σ y : {y : α // y ≠ x}, {z : α // z ≠ x ∧
         ↪  z ≠ y.1} := {
    toFun := fun xyz =>
      ⟨xyz.1.1, ⟨xyz.1.2.1, by simpa [ne_comm] using
       ↪  xyz.2.1⟩,
        ⟨xyz.1.2.2, by simpa [ne_comm] using xyz.2.2.1,
          by simpa [ne_comm] using xyz.2.2.2⟩⟩
    invFun := fun xyz =>
      ⟨(xyz.1, xyz.2.1.1, xyz.2.2.1),
        by simpa [ne_comm] using xyz.2.1.2,
        by simpa [ne_comm] using xyz.2.2.2.1,
        by simpa [ne_comm] using xyz.2.2.2.2⟩
    left_inv := by intro xyz; cases xyz; rfl
    right_inv := by intro xyz; cases xyz; rfl
  }
  have hpair :
      ∀ x : α, ∀ y : {y : α // y ≠ x},
        Fintype.card {z : α // z ≠ x ∧ z ≠ y.1} =
         ↪  Fintype.card α - 2 := by
    intro x y
    have hxy : x ≠ y.1 := by simpa [ne_comm] using y.2
    have hEq :
```

```
        Fintype.card {z : α // z = x ∨ z = y.1} = 2 := by
      simpa [or_comm] using
        (Fintype.card_subtype_eq_or_eq_of_ne (a := x) (b
         ↪  := y.1) hxy)
    have hCompl :
        Fintype.card {z : α // ¬(z = x ∨ z = y.1)} =
          Fintype.card α - Fintype.card {z : α // z = x ∨ z
           ↪  = y.1} :=
      Fintype.card_subtype_compl (fun z : α => z = x ∨ z =
       ↪  y.1)
    simpa [hEq, not_or, ne_comm, and_left_comm, and_assoc]
     ↪  using hCompl
  have hcard : ∀ x : α, Fintype.card {y : α // y ≠ x} =
   ↪  Fintype.card α - 1 := by
    intro x
    calc
      Fintype.card {y : α // y ≠ x}
          = Fintype.card α - Fintype.card {y : α // y = x}
           ↪  := by
              simpa using
                (Fintype.card_subtype_compl (fun y : α => y
                 ↪  = x) :
                  Fintype.card {y : α // ¬y = x} =
                    Fintype.card α - Fintype.card {y : α //
                     ↪  y = x})
      _ = Fintype.card α - 1 := by simp
  have hinner :
      ∀ x : α,
        Fintype.card (Σ y : {y : α // y ≠ x}, {z : α // z ≠
         ↪  x ∧ z ≠ y.1}) =
          (Fintype.card α - 1) * (Fintype.card α - 2) := by
    intro x
    rw [Fintype.card_sigma]
    simp [hpair, hcard]
  rw [Fintype.card_congr e, Fintype.card_sigma]
  simp [hinner, mul_assoc]

end Counting

end Biblioteca.Demonstrations
```